

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

Направление	09.03.02 – Информационные системы и технологии
Профиль	Информационные системы и технологии в бизнесе
Факультет	КТИ
Кафедра	АПУ

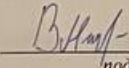
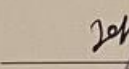
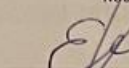
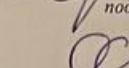
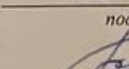
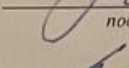
К защите допустить

Зав. кафедрой

Шестопалов М.Ю.

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
БАКАЛАВРА**

**Тема: СИСТЕМА ВЫЯВЛЕНИЯ ДЕГРАДАЦИОННЫХ
ИЗМЕНЕНИЙ В ПАРАМЕТРАХ РАБОТЫ УСТРОЙСТВ
ЖЕЛЕЗНОДОРОЖНОЙ АВТОМАТИКИ**

Студент	 подпись	Волков И.А.	
Руководитель	К.Т.Н., доцент (Уч. степень, уч. звание)	 подпись	Кораблев Ю.А.
Консультанты	ст. преподаватель (Уч. степень, уч. звание)	 подпись	Ерошкин А.В.
	ст. преподаватель (Уч. степень, уч. звание)	 подпись	Скрынская О.А.
	К.Т.Н., доцент (Уч. степень, уч. звание)	 подпись	Белаш О.Ю.
	ассистент (Уч. степень, уч. звание)	 подпись	Ряскова Е.Б.

Санкт-Петербург

2025

**ЗАДАНИЕ
НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ**

Утверждаю

Зав. кафедрой АПУ

_____ Шестопалов М.Ю.

« ____ » _____ 20 ____ г.

Студент Волков И.А.

Группа 1370

Тема работы: Система выявления деградационных изменений в параметрах работы устройств железнодорожной автоматики.

Место выполнения ВКР: Санкт-Петербургский государственный электротехнический университет «ЛЭТИ» им. В.И. Ульянова (Ленина)

Исходные данные (технические требования):

ОС: Windows 7 или выше, ОЗУ: 32 Гб, ЦП: 64-разрядный с тактовой частотой 2 ГГц и выше

Содержание ВКР:

Обзор аналогов, Постановка задачи, Описание формата архивов, Реализация базовых инструментов, Описание решения без использования машинного обучения, Описание решение с использованием машинного обучения, Экономическое обоснование

Перечень отчетных материалов: пояснительная записка, иллюстративный материал.

Дополнительные разделы: Экономическое обоснование ВКР.

Дата выдачи задания
1 апреля 2025 г.

Дата представления ВКР к защите
19 июня 2025 г.

Студент

_____ *В.И. Волков* _____

Волков И.А.

Руководитель

_____ *Ю.А. Кораблев* _____
(Уч. степень, уч. звание)

Кораблев Ю.А.

КАЛЕНДАРНЫЙ ПЛАН ВЫПОЛНЕНИЯ ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ

Утверждаю

Зав. кафедрой АПУ

Шестопалов М.Ю.

«__» _____ 20__ г.

Студент Волков И.А.

Группа 1370

Тема работы: Система выявления деградационных изменений в параметрах работы устройств железнодорожной автоматики.

№ п/п	Наименование работ	Срок выполнения
1	Обзор литературы по теме работы	01.04 – 10.04
2	Постановка задачи	11.04 – 15.04
3	Описание формата архивов	16.04 – 20.04
4	Реализация базовых инструментов	21.04 – 30.04
5	Описание решения без использования машинного	01.05 – 10.05
6	Описание решение с использованием машинного обучения	11.05 – 27.05
7	Экономическое обоснование	28.05 – 29.05
8	Оформление пояснительной записки	30.05 – 03.06
9	Оформление иллюстративного материала	04.06 – 10.06

Студент



Волков И.А.

Руководитель к.т.н., доцент
(Уч. степень, уч. звание)



Кораблев Ю.А.

РЕФЕРАТ

Пояснительная записка 60 стр., 13 рис., 11 табл., 10 ист.

ОБНАРУЖЕНИЕ АНОМАЛИЙ, ЖАТ, СТДМ

В данной работе разработан прототип системы, предназначенной для выявления отклонений в работе устройств ЖАТ на основе алгоритмов машинного обучения. Основная цель исследования заключалась в создании моделей, способных эффективно детектировать деградационные изменения в значениях параметров работы устройств ЖАТ. Предложенный подход основывается на комбинации автоэнкодера и рекуррентной нейронной сети, что позволяет значительно снизить количество ложных срабатываний, соответствуя допустимому порогу в 5%. В процессе тестирования на реальных данных была подтверждена высокая эффективность системы в выявлении отклонений до появления серьезных отказов. Внедрение системы позволит значительно повысить точность диагностики по сравнению с уже разработанными методами.

ABSTRACT

In this paper, we have developed a prototype system designed to detect deviations in the operation of RATMF devices based on machine learning algorithms. The main purpose of the research was to create models capable of effectively detecting degradation changes in the values of the parameters of the harvester devices. The proposed approach is based on a combination of an autoencoder and a recurrent neural network, which significantly reduces the number of false positives, meeting an acceptable threshold of 5%. In the process of testing on real data, the high efficiency of the system in detecting deviations before serious failures occurred. The implementation of the system will significantly improve the diagnostic accuracy compared to the methods already developed.

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ.....	8
ВВЕДЕНИЕ.....	9
1. Обзор аналогов.....	11
1.1. Список выбранных аналогов.....	11
1.2. Описание аналогов.....	11
1.2.1. Аналог 1.....	11
1.2.2. Аналог 2.....	12
1.2.3. Аналог 3.....	13
1.2.4. Аналог 4.....	14
1.3. Выбор критериев и оценивание аналогов.....	14
1.4. Выводы.....	15
2. Постановка задачи.....	16
3. Описание формата архивов.....	17
3.1. Файл полного среза.....	17
3.2. Файл изменений.....	18
3.3. Файл нормалей.....	19
4. Реализация базовых инструментов.....	20
4.1. Реализация библиотеки чтения архивов.....	20
4.2. Реализация визуализатора архива.....	22
5. Описание решения без использования машинного обучения.....	24
5.1. Идея выявления поведения, отличного от нормального.....	24
5.2. Приложение для ручного поиска аномалий.....	25
5.3. Приложение для автоматического поиска аномалий.....	28
6. Описание решение с использованием машинного обучения.....	31
6.1. Выравнивание данных.....	31
6.2. Разбиение данных.....	32
6.2.1. Минутное разбиение.....	32
6.2.2. Разбиение по типу параметра.....	32
6.2.3. Разбиение по типу и группе нормалей.....	33
6.2.4. Динамическое разбиение.....	34
6.3. Создание моделей машинного обучения.....	34
6.3.1. Подготовка данных.....	36

6.3.2.	Классическая искусственная нейронная сеть.....	36
6.3.3.	Автоэнкодер.....	37
6.3.4.	Рекуррентная нейронная сеть и автоэнкодер.....	38
6.3.5.	Оценка результата.....	40
6.4.	Создание прототипа программы анализа.....	40
7.	Экономическое обоснование	50
7.1.	Концепция	50
7.2.	Составление плана выполнения работ.....	50
7.3.	Расчет расходов на оплату труда.....	51
7.4.	Амортизационные отчисления.....	53
7.5.	Расчет материальных затрат.....	54
7.6.	Оценка накладных расходов.....	55
7.7.	Расчет полной себестоимости работы.....	56
7.8.	Вывод.....	56
	ЗАКЛЮЧЕНИЕ	58
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	59

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

Qt – кроссплатформенная программирования C, C++, QML. среда разработки на языках.

Модель – это объект (хранящийся локально в файле), который был обучен для распознавания определенных типов шаблонов.

Устройства ЖАТ – устройства железнодорожной автоматики и телемеханики.

Системы ЖАТ - системы железнодорожной автоматики управляют устройствами сигнализации, централизации и блокировки, а системы телемеханики контролируют их работу.

Фреймворк – это программная платформа, которая предоставляет готовую основу для создания приложений или веб-сайтов, упрощая разработку и ускоряя процесс.

ВВЕДЕНИЕ

На сети железных дорог ОАО «РЖД» применяются системы технической диагностики и мониторинга (СТДМ) устройств железнодорожной автоматики и телемеханики (ЖАТ). СТДМ осуществляет контроль состояния и измерение значений различных параметров работы устройств ЖАТ посредством своего напольного оборудования (датчиков), подключенного к устройствам ЖАТ на станциях и перегонах.

СТДМ решает комплекс задач, одной из которых является контроль исправного состояния и формирование диагностических сообщений при выявлении случаев отклонения от нормальной работы устройств ЖАТ для обслуживающего персонала хозяйства автоматики и телемеханики ОАО «РЖД».

В связи с тем, что отказы в работе устройств ЖАТ приводят к задержкам в движении поездов и большим экономическим потерям, перед СТДМ стоит задача не только выявления отказов, когда значения параметров работы устройств ЖАТ уже вышли за пределы заданных нормативных значений, но и как можно более ранней диагностики предотказных состояний. Поиск таких предотказных состояний предлагается проводить на основе фиксации изменений в динамике изменений значений измеряемых параметров, когда значения измеряемых параметров еще не вышли за предельные нормативные значения, но тенденция и характер графика изменения параметра уже начинает свидетельствовать о наличии деградационных изменений с последующим выходом устройства ЖАТ из работоспособного состояния в неисправное.

В данной работе целью является разработка прототипа системы, которая выявляет изменения или отклонения в режиме работы устройств железнодорожной автоматики на основе анализа значений параметров работы этих устройств, контролируемых системой технической диагностики и

мониторинга АПК-ДК (СТДМ), эксплуатируемой на железных дорогах ОАО «РЖД».

Объект исследования - устройства ЖАТ.

Предмет исследования – эффективность выявления изменений в режиме работы устройств железнодорожной автоматики с использованием алгоритмов машинного обучения для диагностики.

Главным предложенным решением является модель на основе автоэнкодера и рекуррентной нейронной сети ввиду того, что существующие алгоритмы работают только когда значения параметров уже вышли за пределы заданных нормативных значений. Также в существующих подходах не используются алгоритмы машинного обучения, применение которых может существенно повысить эффективность обнаружения аномалий. По этой причине задачами данного исследования являются:

- Анализ существующих методов мониторинга и диагностики железнодорожной автоматики.
- Разработка системы идентификации аномалий и отклонений в режиме работы устройств ЖАТ без использования алгоритмов машинного обучения.
- Разработка системы идентификации аномалий и отклонений в режиме работы устройств ЖАТ с использованием алгоритмов машинного обучения.
- Тестирование обученных моделей на реальных данных.
- Оценка эффективности алгоритмов машинного обучения для диагностики.

1. ОБЗОР АНАЛОГОВ

1.1. Список выбранных аналогов.

В качестве аналогов были выбраны 4 статьи, которые описывают существующие решения подобной задачи:

1. Статья «Решение задачи обнаружения аномалий сетевого трафика с помощью сверточной нейронной сети». [1]
2. Статья «Поиск аномалий с помощью автоэнкодеров». [2]
3. Статья «Предупреждение неисправностей в системах технического диагностирования и мониторинга устройств железнодорожной автоматики и телемеханики». [3]
4. Статья «Применение искусственных нейронных сетей в задачах обнаружения аномалий в поведении сложных динамических объектов». [4]

1.2. Описание аналогов.

1.2.1. Аналог 1.

Главная задача, рассматриваемая в статье – обнаружение сетевых атак на основе имеющегося трафика. Основными методами обнаружения таких атак является метод на основе сигнатур, т.е. поиск признаков уже известных атак по хранящимся в базе данных сигнатурам и шаблонам атак и метод на основе аномалий, т.е. выявление моделей поведения, которые имеют определенное отклонение от базовых параметров, что может сигнализировать о возможных инцидентах безопасности. Упор в статье делается именно на второй метод, т.е. обнаружение аномалий при помощи нейронных сетей.

Также в статье подробно рассматривается процесс подготовки данных и построение модели со вставками кода. Несмотря на то, что автор не указывает фреймворки, которые были использованы при построении модели, можно догадаться, что это sklearn и tensorflow.

В качестве разрабатываемой модели нейронной сети была выбрана сверточная нейронная сеть. В качестве метрик оценки эффективности работы модели выбрали стандартные метрики оценки, а именно: точность, полнота и f1-мера. Точность составила 91%, полнота 100% и f1-мера 95%. Т.е. доля правильных ответов составила 95%, что является хорошим показателем и свидетельствует о качестве и эффективности модели.

По отношению к текущему исследованию можно с уверенностью сказать, что такой подход можно взять на вооружение, т.к. в данной задаче существует вариативность, т.е. не один тип атак, а несколько. В дальнейшем, возможно, это позволит обучить одну модель сразу для нескольких параметров. Очевидно, что единую построить единую модель невозможно, т.к. разные типы параметров имеют абсолютно разное поведение, но такой подход позволит снизить количество моделей, что положительно скажется скорости обучения и на эффективности использования серверных ресурсов.

1.2.2. Аналог 2.

Точно также как и аналоге 1 основной задачей является детектирование аномалий сетевого трафика. Отличием является другой подход. В данной статье авторы предлагают использовать так называемые автоэнкодеры, т.е. нейронные сети прямого распространения.

Также как и в предыдущем аналоге в качестве фреймворков для построения модели были использованы `sklearn` и `tensorflow`.

В статье подробно рассмотрен процесс подготовки, а также один из возможных подходов к анализу уже имеющихся данных. Таким методом стало построение тепловой карты, что позволило выявить высокую зависимость данных между собой. Такие данные удалялись из обучающей выборки для повышения эффективности работы.

Рассматриваемыми подходами к построению автоэнкодеров рассмотрены два варианта: глубокий автоэнкодер и сверточный автоэнкодер. Также для подтверждения эффективности работы таких моделей было

проведено сравнение с двумя управляемыми алгоритмами машинного обучения, а именно с алгоритмом логистической регрессии и методом опорных векторов.

В качестве метрик оценки были взяты полнота, точность, f1-метра и auc – площадь под кривой. В результате было выявлено явное преимущество автоэнкодеров над другими методами машинного обучения.

В рамках данного исследования явно стоит пробовать такой метод, потому что он обладает хорошей точностью, а также позволяет не тратить ресурсы на другие методы ввиду их неэффективности в рамках решения задачи обнаружения аномалий. Возможной проблемой будет отсутствие возможности объединять параметры, что может привести к созданию большого количества моделей, из-за чего потребуется гораздо больше вычислительных ресурсов.

1.2.3. Аналог 3.

В статье подробно описываются причины, по которым системы АПК-ДК (СТДМ) должна оборудоваться собственными средствами анализа параметров, снимаемых на железной дороге.

Также описывается уже имеющееся решение, которое продолжительное время эксплуатируется на железных дорогах ОАО «РЖД» по всей России. Приведен алгоритм определения изменения состояния, описанный на языке разметки XML.

Предложенный алгоритм не использует подходы машинного обучения. Идея алгоритма основана на средних значениях в пределах нормалей. Но, т.к. поведение какого-либо устройства может быть непредсказуемым, то такой алгоритм не является эффективным, что было доказано на практике. Поэтому улучшение имеющегося алгоритма скорее всего не принесет плодов и от него нужно отказаться в пользу подходов и использованием машинного обучения, т.к. в похожих задачах они показывают большую эффективность.

1.2.4. Аналог 4.

В данной статье рассматривается построение одноклассового классификатора на основе классической нейронной сети. Применение схоже с предыдущими рассмотренными источниками, а именно обнаружение аномалий в сети и поведении сложных динамических объектов.

Подробно описано что такое одноклассовая классификация и как она работает, а также параметры, которые влияют на эффективность работы. Применение подобного подхода используется в интеллектуальном анализе на основе технологии Data Mining и определении аномального трафика сети. Также приведены примеры, показывающие эффективность данного подхода в рамках описанной задачи.

Т.к. модель хорошо себя показала в рамках рассматриваемой в статье задачи, можно пробовать применять в ходе текущего исследования. Однако эффективность зависит от множества параметров ввиду особенностей такого подхода. Ключевой особенностью являются высокие аппаратные требования и временные затраты на этапе обучения.

1.3. Выбор критериев и оценивание аналогов.

Т.к. наибольшую эффективность показали подходы, использующие машинное обучение, то ключевым критерием будет *использование машинного обучения* при реализации.

Также на качество и эффективность влияет используемый *алгоритм машинного обучения*, поэтому он будет выделен в отдельный критерий оценки.

За удобство разработки и переносимость решений под разные платформы, а также за совместимость с другими языками программирования отвечают *инструменты разработки*, используемые при построении моделей.

Пригодность алгоритма показывает его *эффективность*, т.е. оценка по определенным метрикам.

Т.к. серверные ресурсы ограничены, и модель, используемая на сервере, может быть заменена на другую, то имеет значение *скорость обучения*.

В таблице 1.1. представлено сравнение аналогов между собой по критерию, которые были выбраны.

Таблица 1.1 – Сравнение аналогов

Критерий	Решение задачи обнаружения аномалий сетевого трафика с помощью сверточной нейронной сети	Поиск аномалий с помощью автоэнкодеров	Предупреждение неисправностей в системах технического диагностирования и мониторинга устройств железнодорожной автоматики и телемеханики	Применение искусственных нейронных сетей в задачах обнаружения аномалий в поведении сложных динамических объектов
использование машинного обучения	Да	Да	Нет	Да
алгоритм машинного обучения	Сверточная нейронная сеть	Автоэнкодер	Машинное обучение не применяется	Классическая искусственная нейронная сеть
инструменты разработки	Sklearn, tensorflow	Sklearn, tensorflow	Неизвестно	Неизвестно
эффективность	95%	94%	Неизвестно	95%
скорость обучения	Неизвестно	7 минут	Обучение не требуется	От нескольких часов до нескольких дней

1.4. Выводы.

На данный момент решения данной конкретной задачи не существует. Поэтому большая часть аналогов, которая связана с поиском аномалий не относится к выбранной теме напрямую, но помогает увидеть возможные подходы к ее решению и использовать опыт, полученный другими исследователями раньше, чтобы получить хороший результат сейчас.

Исходя из описанных аналогов некоторые из предложенных методов будут опробованы на практике, после чего будет произведено сравнение и выбор лучшего подхода.

Таким образом, основной задачей исследования является подготовка данных и обучение различных моделей машинного обучения с целью выявления наиболее эффективного алгоритма решения исходной задачи.

2. ПОСТАНОВКА ЗАДАЧИ

Цель работы: разработать прототип системы, которая выявляет изменения или отклонения в режиме работы устройств железнодорожной автоматики на основе анализа значений параметров работы этих устройств, контролируемых системой технической диагностики и мониторинга АПК-ДК (СТДМ), эксплуатируемой на железных дорогах ОАО «РЖД».

В случае использования машинного обучения, система включает в себя комплекс программного обеспечения, состоящий из 3 программ, которые должны работать независимо друг от друга:

1. Программа, которая преобразует архивы в csv-файлы, используемые для обучения моделей.
2. Программа, обучающая модели.
3. Программа, анализирующая архивы на основе обученных моделей.

В случае, если машинное обучение использоваться не будет, система представляет только программу, анализирующую архивы на основе разработанных правил.

Задачи, решаемые системой:

1. Чтение и преобразование архивов параметров;
2. Обучение и сохранение моделей машинного обучения;
3. Анализ архивов параметров;
4. Визуализация результата анализа;
5. Сохранение результатов анализа;

Среды разработки: Qt Creator 15.0, MS Visual Studio 2022, PyCharm Community Edition 2024.3.1.1.

Программный язык разработки: C++17, Python 3.

Операционная система: Windows 7 или выше.

3. ОПИСАНИЕ ФОРМАТА АРХИВОВ

Сервер формирует несколько различных типов файлов особой структуры, которые потом компонуются с помощью zip-алгоритма (метод сжатия deflate:normal) и публикуются в виде единого файла-набора (далее архива) в указанную в настройках сервера выходную папку с группировкой по суткам.

Каждый архив представляет собой часовой набор, состоящий из 61 файла. Всего в архиве параметров 3 типа файлов: *.main, *.part, *.normals.

Структура каждого файла архива выглядит следующим образом:

- Заголовок архива
- Тело архива

3.1. Файл полного среза

Файл полного среза представляет собой файл внутри архива, имеющий расширение *.main и содержит информацию о всех параметрах на данном участке железной дороги.

Код структуры файла полного среза представлена на листинге 3.1. и 3.2
Листинг 3.1. – Структура изменений параметров в файле полного среза

```
struct MEASURE_HISTORY
{
    /// Идентификатор параметра в рамках места АПК-ДК
    WORD measureID;

    /// Первичного состояние параметра
    struct MEASURE_INIT_VALUE
    {
        /// Время получения значения параметром
        TIME4 time;
        /// Миллисекунды к time
        BYTE millisecs;
        /// Значение параметра
        DWORD value;
        /// Критичность значения
        BYTE critical;
        /// Активная группа нормалей
        BYTE normalGroup;
    } initState;
```

Продолжение листинга 3.1

```
    /// Количество изменений значения параметра
    WORD changesCount;
    /// Список изменений значения
    std::vector<MEASURE_VALUE_CHANGE> changes;
};
```

Листинг 1.2. – Структура, хранящая одно изменение параметра

```
/// Изменение значения параметра
struct MEASURE_VALUE_CHANGE
{
    /// Разница относительно времени начала интервала архива в
    заголовке
    WORD timeOffset;
    /// Миллисекунды к timeOffset
    BYTE millisecs;
    /// Значение параметра
    DWORD value;
    /// Критичность значения
    BYTE critical;
    /// Активная группа нормалей
    BYTE normalGroup;
};
```

3.2. Файл изменений

Файл изменений представляет собой файл внутри архива, имеющий расширение *.part и содержит только изменения параметров. Является дополнительным к файлу полного среза.

Структура файла изменений аналогична структуре файла полного среза за исключением структуры MEASURE_HISTORY. Код измененного фрагмента представлен на листинге 3.3.

Листинг 3.3 – Структура MEASURE_HISTORY для файла изменений

```
struct MEASURE_HISTORY
{
    /// Идентификатор параметра в рамках места АПК-ДК
    WORD measureID;
    /// Количество изменений значения параметра
    WORD changesCount;
    /// Список изменений значения
    std::vector<MEASURE_VALUE_CHANGE> changes;
};
```

3.3. Файл нормалей

Файл нормалей представляет собой файл внутри архива, имеющий расширение *.normals и содержит список параметров и их нормативных значений. Структура файла нормалей представлена на листинге 1.4.

Листинг 1.4. – Структура файла нормалей.

```
struct MEASURE_NORMAL_GROUP
{
    /// Группа нормалей
    BYTE normalGroup;
    /// Количество нормалей в группе
    BYTE normalsCount;
    /// Список значений нормалей
    MEASURE_NORMAL_VALUE* normals;
};

struct MEASURE_NORMAL_VALUE
{
    /// идентификатор нормали
    BYTE normalID;
    /// Значение нормали
    DWORD value;
};
```

4. РЕАЛИЗАЦИЯ БАЗОВЫХ ИНСТРУМЕНТОВ

4.1. Реализация библиотеки чтения архивов.

Архивы параметров имеют особую структуру, для чтения которой необходимо разработать универсальный инструмент для чтения и записи.

Для реализации необходима дополнительная библиотека, позволяющая работать с zip-архивами. В данной работе была использована библиотека libzip, реализованная на языке программирования C.

Общая структура:

- Пространство имен archive – пространство имен для всех классов, связанных с реализацией чтения и записи архивов.
- Класс archive::File – базовый абстрактный класс для всех типов файлов внутри архива.
- Класс archive::MainFile – класс, наследующий класс File и реализующий чтение файла полного среза.
- Класс archive::PartFile – класс, наследующий класс File и реализующий чтение файла изменений.
- Класс archive::NormalsFile – класс, наследующий класс File и реализующий чтение файла нормалей.
- Класс archive::Archive – класс, содержащий данные о всех файлах и реализующий логику их чтения из zip-архива.

Класс File содержит методы read и write, которые читают и записывают файл соответственно. Метод read реализуется при помощи приватных методов readHeader и readData. readHeader читает данные заголовка файла, а т.к. заголовок для всех типов файлов одинаковый, то этот метод реализован в этом же классе File. Метод readData зависит от типа файла, поэтому он является виртуальным, при этом его поведение определяется внутри класса наследника. Запись в файл посредством публичного метода write реализована аналогично. Объявление класса File, его полей и методов представлено в листинге 4.1.

Листинг 4.1 – Объявление класса File

```
class File
{
public:
    File();
    File(BYTE* buf, const uint64_t& size, const std::string&
fileName);
    ~File();

    virtual void read();
    virtual void write();

protected:
    void open(const std::string& fileName);
    void close();

    void readHeader();
    virtual void readData() = 0;

    void writeHeader();
    virtual void writeData() = 0;

private:
    template<class T>
    void readBytes(T& var);

    template<class T>
    void writeBytes(T& var);

    static DWORD toDWORD(const WORD& first, const WORD&
second);

protected:
    ARCHIVE_HEADER      m_header;

    std::string         m_fileName;

private:
    BYTE*               m_buf;
    uint64_t            m_size;
    int                 m_offset;
    std::ofstream       m_fout;
};
```

Класс Archive является базовым классом для работы с архивами параметров. Имеет стандартный набор методов для манипулирования архивом, а именно open, close, read, write. Остальные методы, которые было бы логично включить в этот класс отсутствуют. Это сделано специально, т.к. структура возвращаемых данных может быть разной, поэтому пользователь,

наследовав класс Archive, должен самостоятельно реализовать метод, который будет возвращать данные в определенном формате.

4.2. Реализация визуализатора архива.

При поиске аномалий нельзя опираться на «сухие» значения, полученные из архивов. Для полноценной картины происходящего необходим инструмент, который способен показать, как изменялся график конкретного параметра за выбранный промежуток времени. Интерфейс программы представлен на рисунке 4.1.

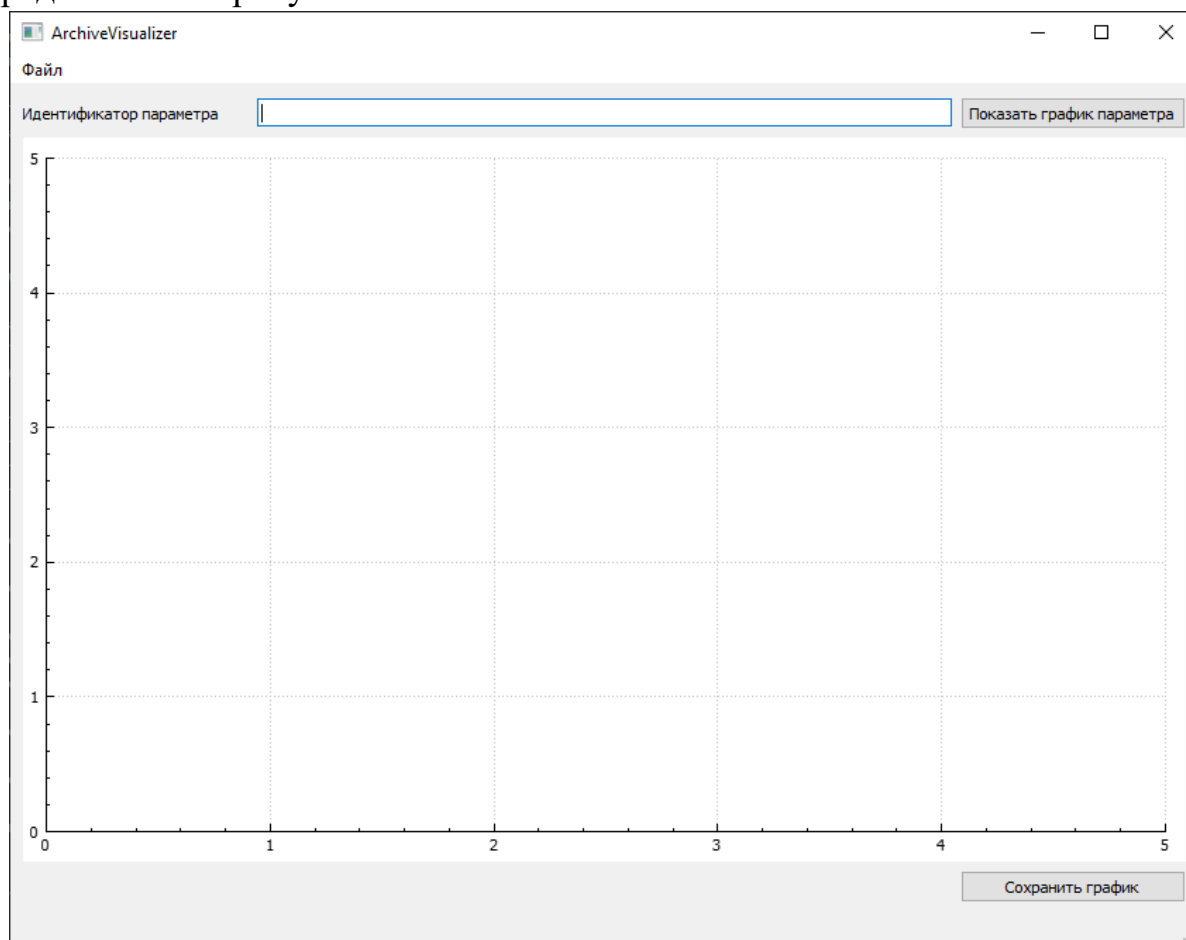


Рисунок 4.1 – Интерфейс программы визуализации архивов

Загрузка файлов архивов производится через меню «Файл» в левом верхнем углу приложения. При нажатии на это меню открывается диалоговое окно проводника, где выбираются архивы, после чего они загружаются в программу.

Последняя версия позволяет загружать любое количество последовательных архивов и показывать историю изменения конкретного

параметра, идентификатор которого введен в поле «Идентификатор параметра». На рисунке 4.2 представлен график параметра с идентификатором 135004161.

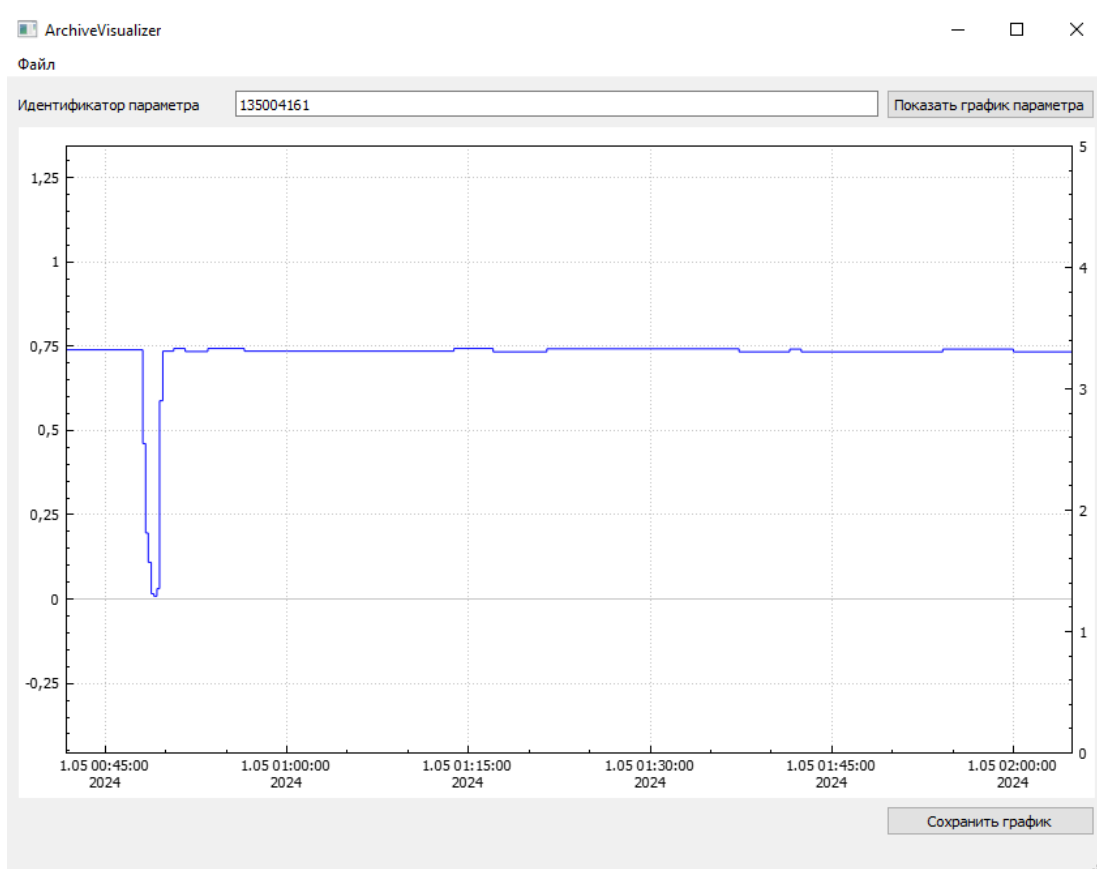


Рисунок 4.2 – График параметра 135004161

Также в программе предусмотрена возможность сохранения изображения. Для этого необходимо нажать на кнопку «Сохранить график» в правом нижнем углу приложения. Откроется диалоговое окно проводника, где можно будет выбрать путь и расширение сохраняемого файла. В последней версии приложения поддерживаются форматы png, jpg, pdf.

5. ОПИСАНИЕ РЕШЕНИЯ БЕЗ ИСПОЛЬЗОВАНИЯ МАШИННОГО ОБУЧЕНИЯ

5.1. Идея выявления поведения, отличного от нормального.

Идея заключается в разбиении графика изменения параметра на определенные временные отрезки и сравнивать их с эталонным. Эталонный отрезок формируется по определенному правилу:

- Изначально за эталон берется значение в момент T_0 ;
- Если за рассматриваемый момент значение изменилось и сохраняет свое состояние за время $3T$, оно принимается за эталонное.

Если графики изменения параметра сильно отличаются, то можно говорить об аномальном поведении параметра.

Если рассматривать каждое изменение параметра как случайную величину, то можно воспользоваться выборочным линейным коэффициентом парной корреляции Пирсона [5], который оценивает тесноту линейной корреляционной зависимости между двумя величинами (в нашем случае, форму графиков). На выходе эта зависимость выражается числом с плавающей точкой в диапазоне $[-1;1]$.

По шкале Чеддока значения коэффициента соответствуют:

- 0 - 0.1 - очень слабая зависимость или ее отсутствие;
- 0.1 - 0.3 - слабая зависимость;
- 0.3 - 0.5 - умеренная зависимость;
- 0.5 - 0.7 - заметная зависимость;
- 0.7 - 0.9 - сильная зависимость;
- 0.9 - 0.99 - очень сильная зависимость;
- 0.99 - 1.0 – функциональная;

Если коэффициент отрицательный, то зависимость обратная, а если положительный, то прямая.

5.2. Приложение для ручного поиска аномалий.

Для проверки работоспособности данного метода было реализовано приложение, которому на вход дается архив параметров, а также уникальный идентификатор параметра, график которого необходимо построить. На рисунке 5.1 показано, как выглядит часть графика для параметра 548536456.

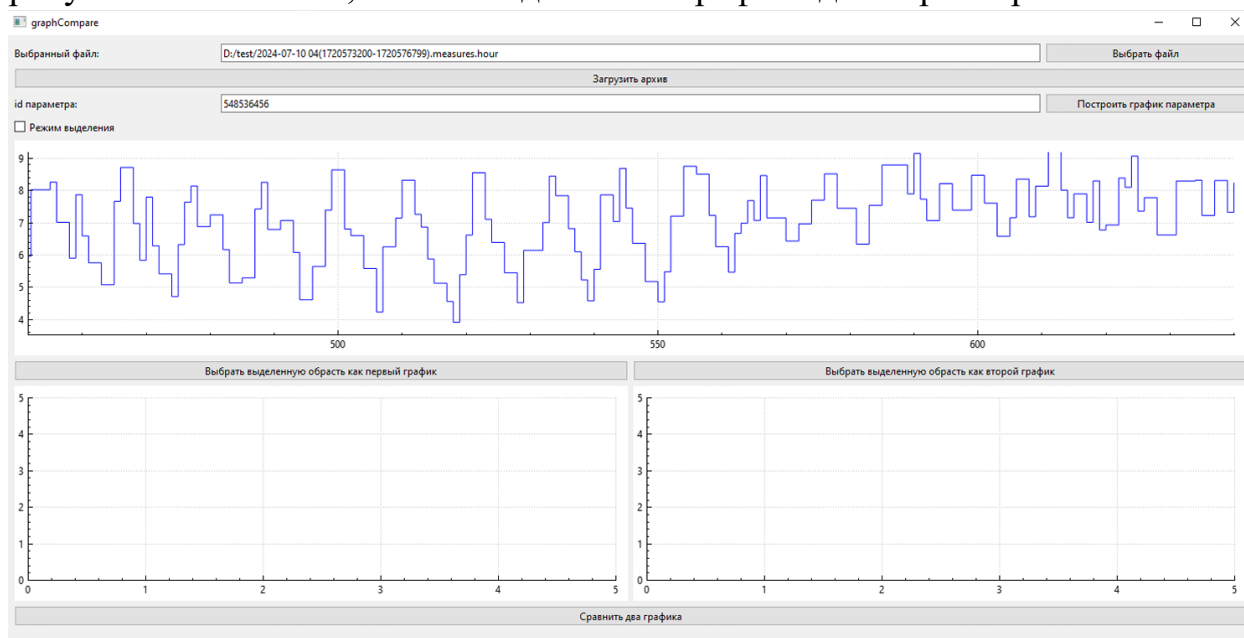


Рисунок 5.1 – Фрагмент графика параметра 548536456

После построения графика определенного параметра можно выделять область и сохранять ее в виде отдельного графика. Всего можно сохранить два новых графика, а после чего сравнить их.

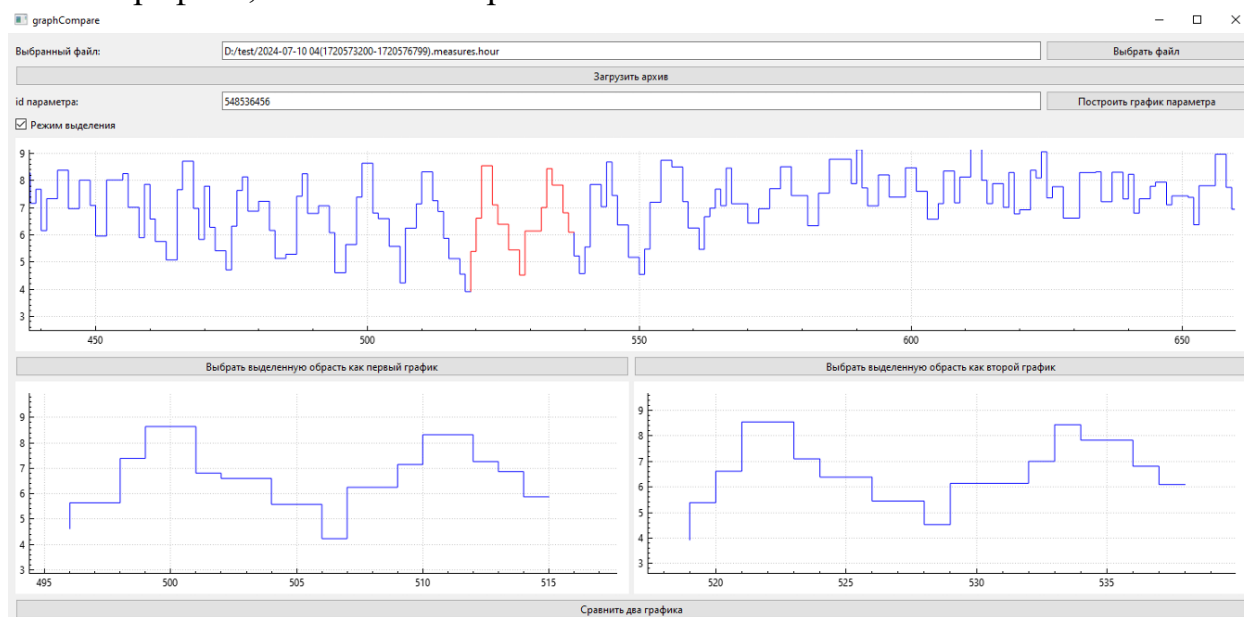


Рисунок 5.2 – Выбор произвольных областей графика

При нажатии кнопки «Сравнить два графика» будет рассчитан коэффициент корреляции. Если зависимость будет сильной, то программа выдаст, что графики похожи.

На рисунке 5.3 графики отличаются, но незначительно, поэтому можно сказать, что они похожи, в то время как на рисунке 5.4 графики существенно отличаются и сказать, что они похожи нельзя. Коэффициенты корреляции и выводы соответствуют увиденному.

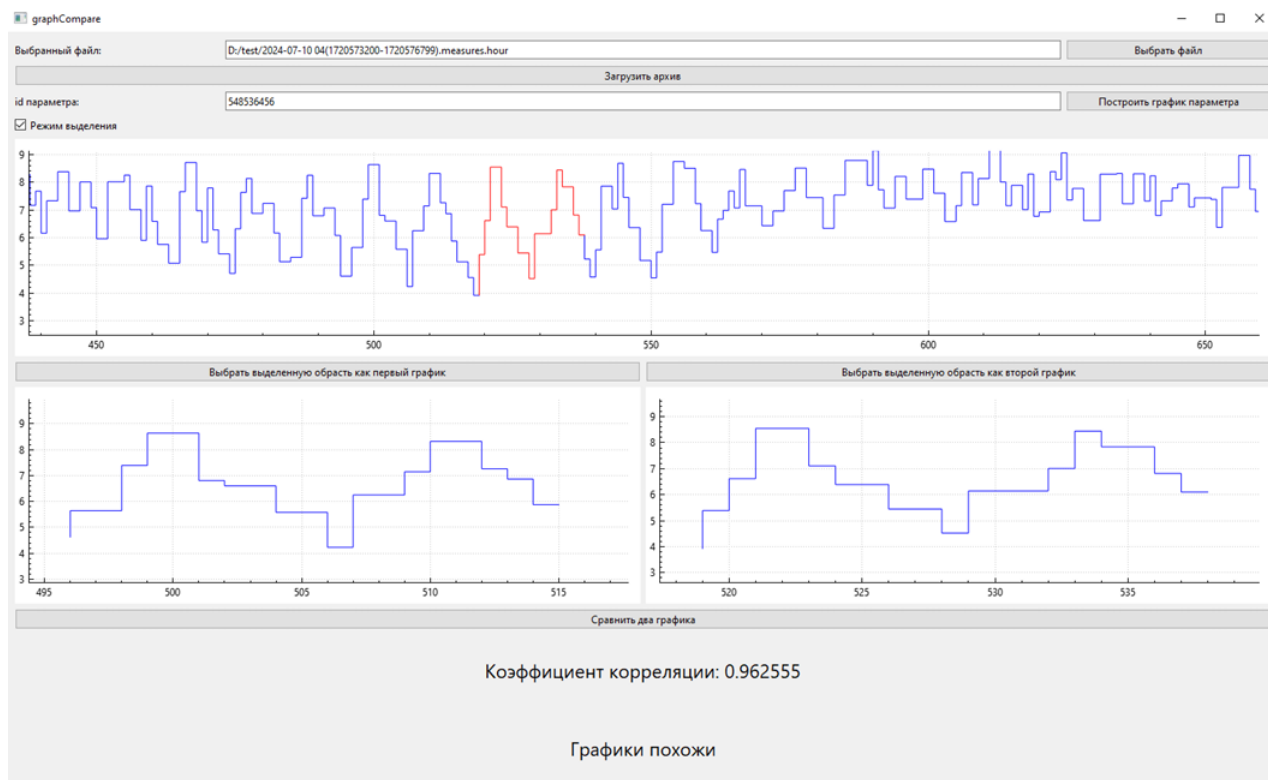


Рисунок 5.3 – Похожие по форме графики

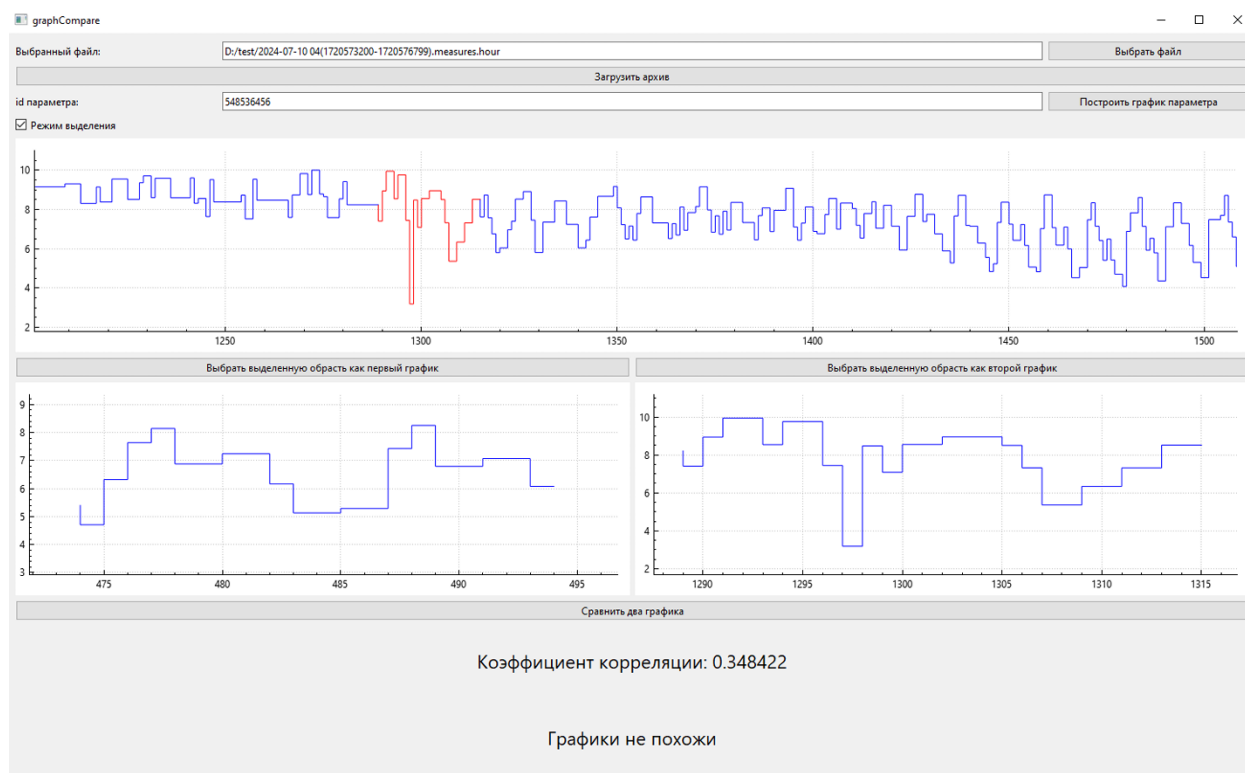


Рисунок 5.4 – Непохожие по форме графики

Т.к. вручную проверять долго, то данное приложение было модифицировано, чтобы работать не с одним архивом, а сразу с папкой архивов, например, одного сервера. Оно разбивает графики изменения параметров на определенные (обязательно равные) отрезки по определенному правилу и сравнивает их с эталонным. Как будет формироваться эталонный отрезок описано в п. 5.3. Если графики оказались сильно непохожими, то это будет записано как аномалия. Для аномалий будет вестись отдельный журнал, в который будет записываться: название архива, два графика (эталонный и аномальный). Журнал представляет собой папку, в которой будут файлы, содержащие информацию об аномалии. Также в приложении дополнительно реализована возможность просмотра аномалий. После анализа всех параметров для пользователя доступен выпадающий список, где можно будет выбрать аномалию и визуально оценить поведение параметра с эталоном. Также будет выведен коэффициент корреляции.

5.3. Приложение для автоматического поиска аномалий.

Правила разбиения графика на отрезки:

Графики нельзя нарезать случайным образом, потому что значения параметров сильно отличаются в разных ситуациях. Это выражается в значении критичности, т.е. насколько значения отличается от нормативных. Для правильного разбиения были разработаны правила, по которым программа делит исходный график для сравнения его частей:

1. Анализ графиков возможно производить, только при нахождении параметра в пределах нормы.
2. Для рассмотрения графика предусмотреть уровень дискретизации (T) в 10 минут. Предусмотреть данный параметра адаптивным.
3. Брать за эталонное значение, значение, полученное в течении 3 периодов. В случае начально работы за эталон берется значение в момент T_0 .
4. Если за рассматриваемый момент значение изменилось и сохраняет свое состояние за время $3T$, оно принимается за эталонное.

На основе этих правил была модифицирована программа, ручного выявления аномалий.

Первая версия этой программы принимала на вход архив, анализировала его и показывала аномалии. Однако такой подход оказался неэффективным, и программа выдавала около 60000 аномалий с одного часового архива, что очень много и, по результатам анализа, неправильно.

Поэтому было принято решение перед анализом архивов производить из конвертирование, чтобы убрать незначительные изменения параметров. Погрешность изменений составляет 5%, но для конвертирования было выбрано 2%. Т.е. если последнее полезное и текущее изменение отличаются друг от друга менее чем на 2%, при этом критичность и группа нормативных значений остались неизменными, то текущее изменение выбрасывалось из архива.

Конечный интерфейс программы представлен на рисунке 5.5. Конвертирование архивов можно делать, а можно и не делать, это остается на выбор пользователю. После анализа, в выпадающий список будут загружены все аномалии, которые были найдены. Их можно посмотреть, нажав на соответствующую кнопку.

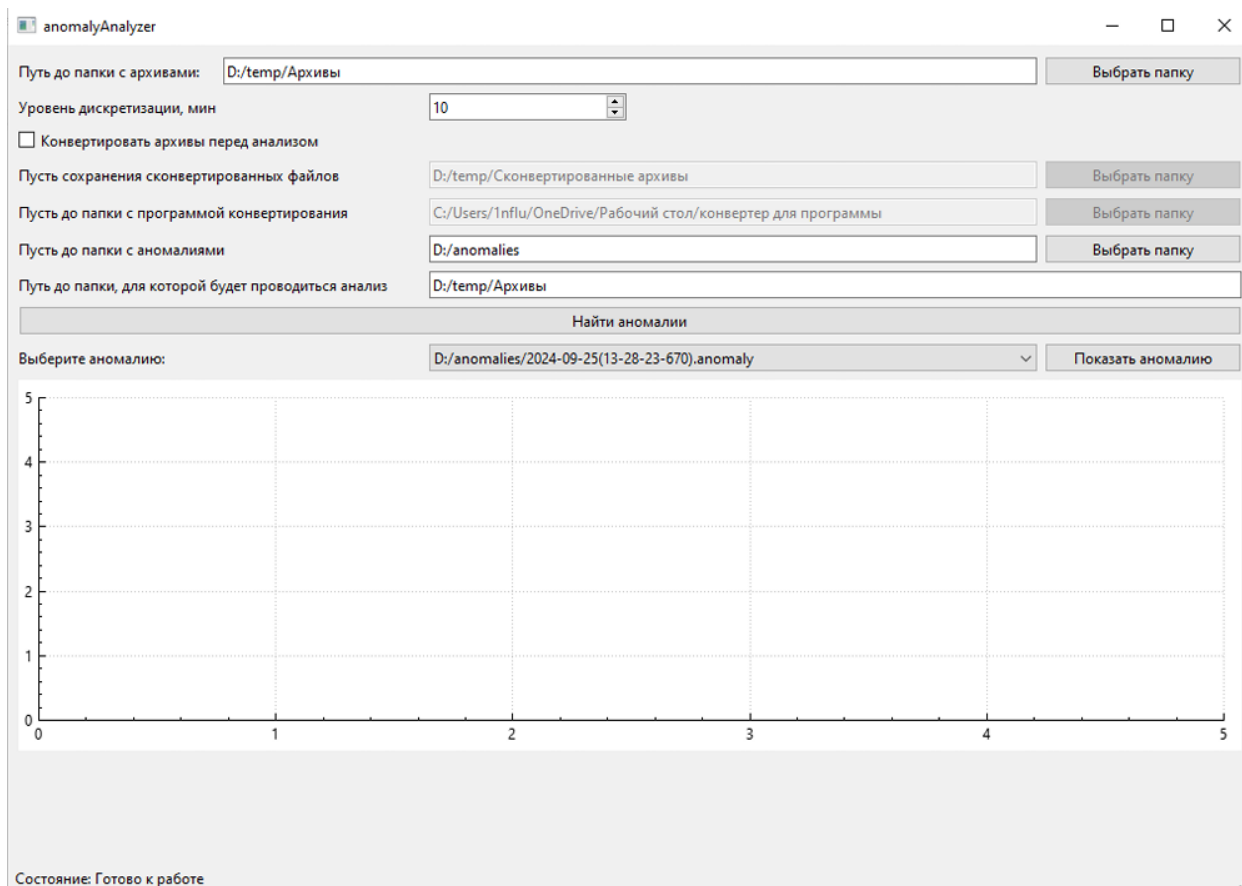


Рисунок 5.5 – Интерфейс программы

Как видно из рисунка 5.6, графики действительно сильно отличаются, нельзя сказать, что они похожи, но, тем не менее, это не является аномалией, т.к. устройство работало в нормальном режиме. Даже после конвертирования количество аномалий сократилось незначительно, примерно на 5-10%, что остается плохим результатом, т.к. большая часть аномалий были выявлены неправильно.

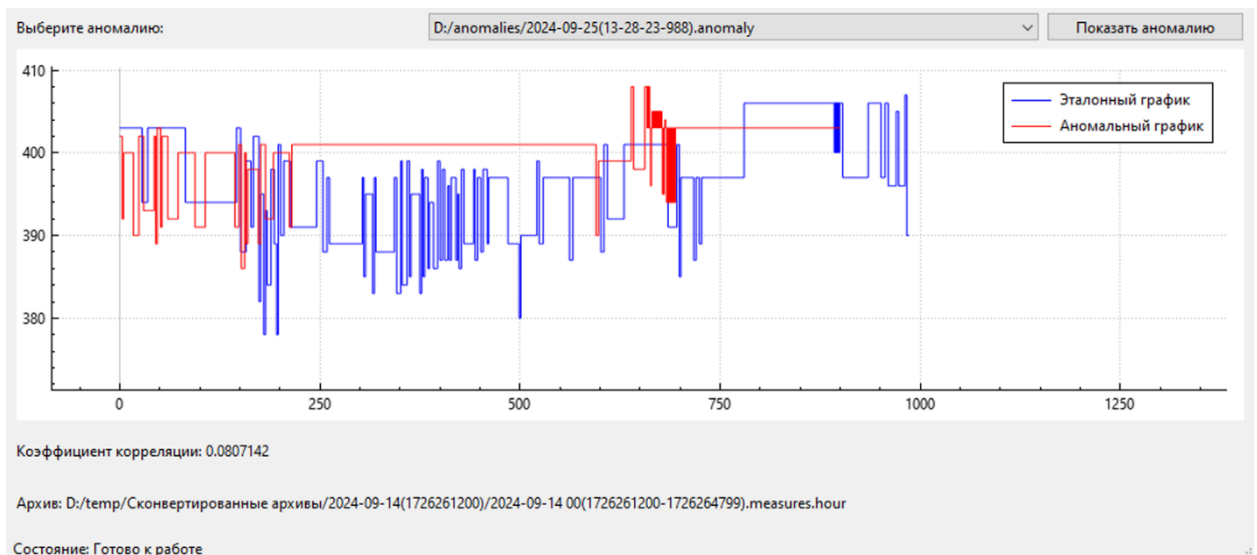


Рисунок 5.6 – Найденная в архиве аномалия

Такой подход показал свою неэффективность, поэтому работа над ним была прекращена. В главе 6 будет рассмотрен другой подход, а именно использование алгоритмов машинного обучения, который показывает хорошие результаты в подобных задачах.

6. ОПИСАНИЕ РЕШЕНИЕ С ИСПОЛЬЗОВАНИЕМ МАШИННОГО ОБУЧЕНИЯ.

Прежде чем обучать модели необходимо правильно подготовить данные. Для этого нужно понимать как устроено хранение данных в архиве. Исходя из листинга 1.2 понятно, что изменение привязано ко времени и лишние значения не хранятся в целях экономии места. В одном файле архива хранятся изменения за 1 минуту, при этом изменения записываются не чаще чем 1 раз в секунду. Соответственно, максимальное количество изменений в файле архива – 60, а минимальное – 0.

6.1. Выравнивание данных.

В действительности количество изменений всегда разное, и такое неравномерное распределение мешает при подготовке данных.

В данной работе проблема была решена следующим образом:

1. Создается массив на 60 значений.
2. Заполняются известные значения (timeOffset в структуре хранения одного изменения является индексом массива)
3. Пустые места заполняются уже известными значениями.

Выше описан общий подход к решению проблемы, но также есть частные случаи, например, когда изменения начинаются не с 0 индекса, а, например, с 20 и первые 20 ячеек нечем заполнять. В таком случае в файле полного среза хранится т.н. инициализационное значение, которым будут заполняться первые N ячеек первого массива изменений. Также последнее изменение сохраняется для каждого параметра. Оно будет инициализационным значением уже для файлов изменений. Это позволяет избежать пропусков значений и не искажать данные, т.к. если нет изменения – значит значение не изменилось. Минус такого подхода в большем количестве памяти, затрачиваемом на хранение, но т.к. архивы всегда читаются последовательно, это не является критичным.

6.2. Разбиение данных.

Для записи данных в csv-файл и последующим использованием при обучении моделей машинного обучения необходимо их разбить на равные отрезки, т.к. называемые вектора или ансамбли.

Для решения данной задачи было разработано несколько подходов.

6.2.1. Минутное разбиение.

Первый вариант – разбиение по минуте, т.е. данные записываются в csv-файл ровно так, как данные записаны в архивах. В выходной папке создается столько файлов, сколько различных параметров входит в архив.

У такого подхода есть существенный недостаток – при обучении будет создано большое количество моделей, каждая из которых будет занимать определенное место в оперативной памяти. Это сильно усложнит разработку системы, т.к. для одного сервера при 5000 различных параметров потребуется примерно 200 ГБ оперативной памяти только для хранения моделей. Поэтому такой подход использовался только в начале работы, далее уже разрабатывались методы оптимизации данных и сокращения количества моделей.

6.2.2. Разбиение по типу параметра.

Следующим подходом было разбиение по типам параметров. Для того, чтобы сократить количество моделей было реализовано разбиение по типам параметров.

Основные типы параметров на сервере:

- Вход путевого приемника
- Выход путевого приемника
- График тока перевода стрелки
- Перекрытие светофоров
- Полюс питания
- Путевой генератор

- Фидер

Также возможны и другие типы, но они встречаются реже.

Идея разбиения заключается в формировании csv-файлов, содержащих данные сразу нескольких параметров. Такое разбиение основано на идее одинакового поведения для одинаковых типов. В дополнение к этому в этом и всех последующих подходах данные нормализовались по формуле 6.1 чтобы убрать большую разницу в значениях.

$$X_{norm} = \frac{X - X_{max}}{X_{max} - X_{min}} \quad (6.1)$$

В выходной папке структура будет выглядеть следующим образом:
Название папки – тип параметра. Внутри папки хранится 2 файла:

- Data.csv – файл в котором хранятся данные для обучения моделей
- Ids.txt – файл, в котором хранятся идентификаторы параметров для которых собраны данные в файле Data.csv

Такой подход позволяет сократить количество моделей до минимума и, соответственно, сократить потребление оперативной памяти на сервере.

6.2.3. Разбиение по типу и группе нормалей.

Третьей вариацией разбиения данных реализовано разбиение по типам и группам нормалей.

У каждого параметра есть группа нормалей, которая обязательно содержит минимальное и максимальное значение. На основе этих значений реализовано данное разбиение. Название папки формируется по следующему правилу: Тип параметра ({группа нормалей}), где {группа нормалей} – диапазон значений нормалей для параметров, записанных в виде «Минимальное значение – максимальное значение». При формировании обучающих данных в папку будут попадать только те параметры, которые соответствуют типу и группе нормалей, для других параметров будут создаваться новые папки. Структура файлов, хранимых в папке аналогична пункту 6.2.2.

Недостаток такого разбиения – обучающие вектора всегда имеют размерность 60, что может являться недостаточным для обучения, т.к. больше количество ключевых фрагментов будет разбито на разные ансамбли.

6.2.4. Динамическое разбиение.

Последним предложенным вариантом разбиения данных является разбиение, аналогичное пункту 6.2.3 с возможностью динамически изменять размерность обучающих векторов. Т.е. программа будет записывать в csv-файл вектора не по 60 значений, а по $t * 60$, где t – количество минут, которое должно попасть в обучающий ансамбль. Это позволит охватить больше ситуаций в одном обучающем векторе, что должно повысить качество обучения. Структура хранения аналогична пунктам 6.2.2 и 6.2.3, но в добавок к двум записанным файлам добавляется файл mins.txt, которых хранит количество минут в одном обучающем векторе.

Существенным недостатком является потерянные значения. Под этим подразумевается ситуация, когда параметр не изменялся определенное количество минут и из архива получено меньшее количество векторов размерности 60. В таком случае, когда количество таких векторов не кратно t , последние вектора не будут записаны в обучающую выборку.

Для решения данной проблемы реализовано смещение на 1 минуту. Например, задано разбиение на 5 минут. Первый вектор будет содержать следующие значения: $X_1, X_2, \dots, X_{300}$. Второй вектор будет содержать значения: $X_{61}, X_{62}, \dots, X_{360}$. Третий вектор: $X_{121}, X_{122}, \dots, X_{420}$, и так далее.

Минус такого подхода в увеличении объема данных на диске и увеличении времени обучения моделей.

6.3. Создание моделей машинного обучения.

Основная задача построенных моделей – распознавание аномалий в режимах работы устройств железнодорожной автоматики и телемеханики. Эта задача является задачей классификации. За основную идею взято обучение

только на положительных примерах. Т.е. в обучающей выборке не должно встречаться аномалий, в крайнем случае, их количество должно быть минимальным.

При создании моделей машинного обучения были использованы 3 подхода:

- Классическая искусственная нейронная сеть
- Автоэнкодер
- Комбинация рекуррентной нейронной сети и автоэнкодера.

Для обучения использовался язык программирования python3, а также фреймворк tensorflow версии 2.10. Дополнительные зависимости: библиотека Pandas и Numpy.

Tensorflow позволяет сохранять обученные модели, а также загружать их, используя разные языки программирования, в том числе C++. При разработке программы визуализации использовалась обертка над Tensorflow API – cppflow.

Обученные модели представляют собой папку, внутри которой находится вся необходимая информация о модели. Это позволяет удобно использовать данный фреймворк при разработке.

В качестве проверки в системе АПК-ДК (СТДМ) был найден архив, который содержал аномалию. Данная аномалия представлена на рисунке 6.1.

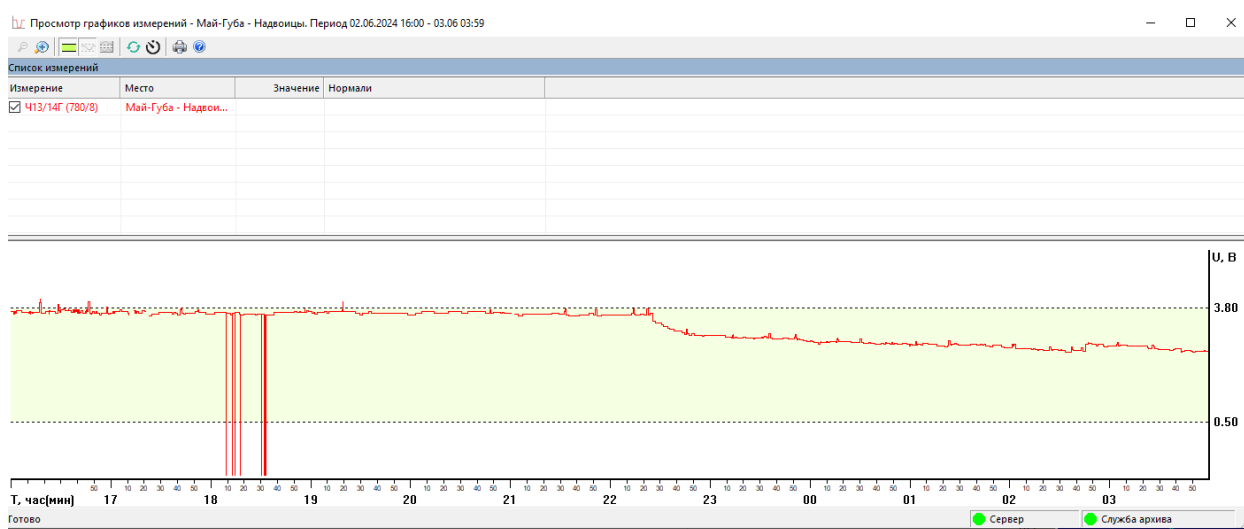


Рисунок 6.1 – Найденная аномалия в системе АПК-ДК (СТДМ)

На рисунке 6.1 изображен параметр с идентификатором 139919407. Тип параметра – путевой генератор. Прослеживается падение напряжения, что свидетельствует о выходе из строя генератора напряжения.

Все модели первоначально проверялись на этом архиве, а также на архиве, который не содержит аномалии

6.3.1. Подготовка данных.

Данные, как уже было сказано ранее, хранятся в csv-файлах. Для их чтения использовалась библиотека pandas. Также в момент чтения данные очищались от пропущенных, пустых и бесконечно больших значений, т.к. алгоритмы машинного обучения плохо с ними работают или не работают вовсе. Код функции чтения и очистки данных представлен в листинге 6.1.

Листинг 6.1 – Функция чтения и очистки данных

```
def get_input_data(path):
    raw_data = 0
    try:
        raw_data = pd.read_csv(path)
    except FileNotFoundError:
        print("File not found.")
    except pd.errors.EmptyDataError:
        return pd.DataFrame([])
    raw_data = raw_data.select_dtypes(exclude=['object'])
    raw_data = raw_data.dropna()
    raw_data = raw_data.replace([np.inf, -np.inf],
np.nan).dropna(axis=1)
    data = np.array(raw_data.values)
    return data
```

6.3.2. Классическая искусственная нейронная сеть.

Для искусственной нейронной сети были заданы следующие параметры: $n = 60 * t$ входных нейронов, где n – количество входных нейронов, а t – количество минут в обучающей выборке. $h = 0.25 * n$, где h – количество скрытых нейронов, а также 2 нейрона в выходном слое. Первый выходной нейрон соответствует аномалии, а второй отсутствию аномалии. Функции активации: сигмоидальная для нейронов скрытого слоя и линейная для нейронов выходного слоя.

Результаты работы классической нейронной сети представлены в таблице 6.1.

Таблица 6.1 – Результат работы классической нейронной сети

Подход к подготовке данных	Объем данных, Гб	Время обучения, мин	Количество найденных аномалий	Количество ложных срабатываний	Количество верно определенных ситуаций
Минутное разбиение	31,9	10	0	0	0
Разбиение по типу параметра	31,9	10	0	0	0
Разбиение по типу параметра и группе нормалей	31,9	10	0	0	0
Динамическое разбиение	90,3	34	0	0	0

Как видно из таблицы 6.1 построенная искусственная нейронная сеть не справилась с задачей обнаружения аномалий, поэтому необходимо использовать другие подходы для данной задачи.

6.3.3. Автоэнкодер.

Автоэнкодер – это нейронная сеть, которая состоит из двух основных частей: кодировщик и декодировщик. Обучение происходит без учителя, поэтому для такого типа нейронной сети не требуются размеченные данные. В процессе обучения получая входные данные кодировщик пытается сжать их до меньшего количества признаков, после чего самостоятельно декодировать их с минимальным искажением. Серьезное отклонение от входного вектора может говорить об аномалии. Такой подход должен хорошо справляться с поставленной задачей.

Построение автоэнкодера представлено в листинге 6.2. Данные сжимаются сначала до $n / 2$ признаков, где n – количество входных нейронов. После первого сжатия данные сжимаются еще в два раза, после чего декодер пытается их восстановить. Функции активации: ReLU для кодировщика и сигмоидальная для декодера. Функция ошибки: среднее квадратичное отклонение.

Листинг 6.2 – Построение автоэнкодера

```
def build_autoencoder(input_dim):  
  
    input_layer = layers.Input(shape=(input_dim,))  
    encoded = layers.Dense(input_dim / 2,  
activation='relu')(input_layer)  
    encoded = layers.Dense(input_dim / 4,  
activation='relu')(encoded)  
  
    decoded = layers.Dense(input_dim / 2,  
activation='relu')(encoded)  
    decoded = layers.Dense(input_dim,  
activation='sigmoid')(decoded)  
  
    autoencoder = models.Model(input_layer, decoded)  
    autoencoder.compile(optimizer='adam', loss='mse')  
  
    return autoencoder
```

Результаты работы модели представлены в таблице 6.2.

Таблица 6.2 – Результат работы автоэнкодера

Подход к подготовке данных	Объем данных, Гб	Время обучения, мин	Количество найденных аномалий	Количество ложных срабатываний	Количество верно определенных ситуаций
Минутное разбиение	31,9	8	50	49	1
Разбиение по типу параметра	31,9	12	42	41	1
Разбиение по типу параметра и группе нормалей	31,9	12	13	12	1
Динамическое разбиение	90,3	46	8	7	1

Как видно из результатов работы, модель смогла определить аномалию, но присутствует большое количество ложных срабатываний. Пользуясь визуализатором архивов, было выяснено, что такое количество ложных срабатываний в основном наблюдается у нестабильных параметров, у которых большой разброс значений.

6.3.4. Рекуррентная нейронная сеть и автоэнкодер.

Рекуррентная нейронная сеть — это тип нейронной сети, который используется для обработки последовательных данных, например, временных рядов. В отличие от классических нейронных сетей, рекуррентные нейронные

сети могут работать с последовательностями данных, что делает их полезными для задач, где важен порядок входных данных.

Рекуррентная нейронная сеть обладает скрытым состоянием, которое сохраняет информацию о предыдущих шагах. Скрытое состояние обновляется на каждом шаге в зависимости от входных данных и предыдущего скрытого состояния. Это позволяет учитывать предыдущие значения и комплексно воспринимать входные данные.

При создании модели часто встречалось такое явление, как градиентный взрыв. Это проблема возникает, когда величина градиентов увеличивается экспоненциально во время обратного распространения ошибки по сети. Для ее решения использовалась модификация классической рекуррентной нейронной сети, которая называется LSTM. Эта модификация позволила работать с долгосрочными зависимостями, избегая градиентных взрывов.

Изменения значений можно рассматривать как временной ряд, поэтому такой подход в совокупности с автоэнкодером должен хорошо себя проявить в задаче обнаружения аномалий.

Результат работы комбинации рекуррентной нейронной сети и автоэнкодера представлен в таблице 6.3.

Таблица 6.3 – Результат работы рекуррентной нейронной сети и автоэнкодера

Подход к подготовке данных	Объем данных, Гб	Время обучения, мин	Количество найденных аномалий	Количество ложных срабатываний	Количество верно определенных ситуаций
Минутное разбиение	31,9	30	20	19	1
Разбиение по типу параметра	31,9	42	9	8	1
Разбиение по типу параметра и группе нормалей	31,9	56	1	0	1
Динамическое разбиение	90,3	364	2	1	1

Как видно из результатов такая комбинация отлично справилась с задачей обнаружения аномалии. При этом динамическое разбиение показало результат хуже, чем разбиение по типу параметра и группе нормалей.

6.3.5. Оценка результата.

В таблице 6.4 представлено сравнение лучших показателей всех видов нейронных сетей, рассмотренных в данной работе. На основе этой таблицы будет выбираться лучший вариант, который будет взят за основу для создания программы, идентифицирующей аномалии. Эта программа будет устанавливаться на сервер и уведомлять о неисправности или скором выходе из строя устройства железнодорожной автоматики и телемеханики.

Таблица 6.4 – Сравнение лучших результатов

Модель машинного обучения	Объем данных, Гб	Время обучения, мин	Количество найденных аномалий	Количество ложных срабатываний	Количество верно определенных ситуаций
Классическая нейронная сеть	31,9	10	0	0	0
Автоэнкодер	90,3	46	8	7	1
Комбинация рекуррентной нейронной сети и автоэнкодера	31,9	56	1	0	1

Лучший результат из всех показала комбинация рекуррентной нейронной сети и автоэнкодера, поэтому именно такие модели будут строиться для всех параметров.

6.4. Создание прототипа программы анализа.

Прототип программы – это приложение с графическим интерфейсом пользователя, которое загружает обученные модели и анализирует с их помощью архивы параметров.

При разработке использовался язык программирования C++(17 стандарт), фреймворк Qt (версия 5.15.2), а также фреймворк машинного обучения Tensorflow (версия 2.10).

Основными модулями приложения являются:

- Analyzer – класс который анализирует архивы параметров, используя загруженные модели.

- TensorflowModelLoader – загрузчик моделей, обученных при помощи фреймворка Tensorflow.
- NsiLoader – загрузчик файлов нормативно справочной информации.

Код заголовочных файлов класса Analyzer, TensorflowModelLoader и NsiLoader представлен в листингах 6.3, 6.4 и 6.5 соответственно.

Листинг 6.3 – Код заголовочного файла класса Analyzer

```
class Analyzer : public QObject
{
    Q_OBJECT

public:
    Analyzer(QObject* parent = nullptr);

signals:
    void started();
    void finished(const std::vector<Anomaly>& anomalies,
                  const std::string& archivePath);

public slots:
    void analyze(archive::MeasureChanges& data,
                 neuralnetwork::Models* models,
                 const std::string archivePath,
                 AnomaliesCountMap* map,
                 MeasuresId* measuresId);

private:
    double mse(const std::vector<float>& f,
               const std::vector<float>& s);

private:
    const double c_invalidMse = -1.0f;
};
```

Класс Analyzer анализирует архивы используя обученные модели. В программе создается один объект класса Analyzer. Этот объект находится в отдельном потоке, чтобы в момент анализа избежать блокировки интерфейса пользователя и позволить просматривать аномалии параллельно с анализом. Процесс выявления аномалий начинается при появлении сигнала analyze, испускаемым главным окном приложения. Анализ архивов выполняется в методе analyze, который принимает 5 параметров:

- Данные в необходимом формате;
- Обученные модели;

- Путь до файла архива;
- Структуру AnomaliesCountMap необходимую для сбора статистики;
- Идентификаторы параметров.

После завершения анализа архива испускается сигнал `finished`, который передает найденные аномалии в архиве. Аномалии отображаются в главном окне приложения. Сразу после завершения анализа одного архива, начинается анализ следующего.

Листинг 6.4 - Код заголовочного файла класса `TensorflowModelLoader`

```
class TensorflowModelLoader : public QObject
{
    Q_OBJECT

public:
    TensorflowModelLoader(QObject* parent = nullptr);
    ~TensorflowModelLoader();

signals:
    void loadStarted();
    void loadFinished(neuralnetwork::Models* models);

public slots:
    void load(const std::string& path);
    void clear();

public:
    Models getModels() const;

private:
    std::vector<DWORD> getIds(const std::string& path)
const;
    int getMins(const std::string path) const;
    threshold getThreshold(const std::string& path)
const;

private:
    Models m_models;

private:
    const std::string c_idsFilename{"ids.txt"};
    const std::string c_minsFilename{"mins.txt"};
    const std::string
c_thresholdFileName{"threshold.txt"};
    const std::string
c_targetFileName{"saved_model.pb"};
};
```

Класс `TensorflowModelLoader` предназначен для загрузки обученных моделей. По аналогии с классом `Analyzer` создается один объект и переносится в другой поток, т.к. загрузка моделей для одного сервера занимает продолжительное время. Загрузка моделей инициируется сигналом `loadModels`, выпускающимся классом главного окна приложения. Загрузка моделей происходит в методе `load`, который принимает путь до папки с моделями. Путь обходится рекурсивно. Модели ищутся по т.н. целевым файлам. Такими файлами являются:

- `ids.txt` – файл, хранящий идентификаторы параметров, для которых обучена модель;
- `mins.txt` – файл, хранящий количество минут, за которые подаются данные в модель;
- `threshold.txt` – файл, хранящий пороговое значение, превышение которого идентифицируется как аномалия;
- `saved_model.pb` – файл, который хранит информацию о сохраненной модели.

При отсутствии одного из этих файлов модель не загружается, т.к. нельзя однозначно определить, как ее использовать в программе. По окончании загрузки модели передадутся в сигнале `finished` для последующего использования.

Для вывода подробной информации о найденных аномалиях необходимы файлы нормативно справочной информации, которые представляют собой файлы Excel, и которые содержат следующие данные:

- Идентификатор параметра;
- Название параметра;
- Название типа параметра;
- Название дороги;
- Номер ШЧ;
- Станцию;

- Идентификатор объекта привязки
- Объект привязки
- Идентификатор типа объекта привязки
- Наименование типа объекта
- Идентификатор типа измерения

Эта информация даст полное понимание про параметр, что облегчит визуальный анализ результатов человеком.

За чтение файлов нормативно справочной информации отвечает класс NsiLoader.

Листинг 6.5 - Код заголовочного файла класса NsiLoader

```
class MeasureTypes : public QObject
{
    Q_OBJECT

public:
    MeasureTypes(QObject* parent = nullptr);

signals:
    void started();
    void finished(const MeasureInfo & types);

public slots:
    void load(const std::string& pathToFolder);
    void clear();

    std::string getTypeById(const DWORD& id);

private:
    MeasureInfo m_measureInfo;
};
```

Загрузка моделей инициируется сигналом loadNsi, выпускающимся классом главного окна приложения. Загрузка моделей происходит в методе load, который принимает путь до папки с файлами НСИ. Путь обходится рекурсивно. Для чтения Excel документа используется библиотека libxl, которая позволяет обрабатывать до двух миллионов строк в секунду. По окончании загрузки модели предадутся в сигнале finished для последующего использования.

На рисунке 6.2 представлен интерфейс программы анализа. Пронумерованы ключевые элементы взаимодействия с программой:

1. Меню файл – в меню можно выбрать модели и файлы НСИ для загрузки, а также удалить их. Модели и файлы НСИ должны лежать в отдельных папках, предназначенных для их хранения;
2. Путь до папки с архивами – путь обходится рекурсивно, отбирая только файлы архивов (measures.hour);
3. Выбор диапазона анализа – в тестовую выборку попадут только те архивы, которые удовлетворяют выбранному времени. Формат даты: ММ-ДД-ГГГГ;
4. Список аномалий – в это поле будут выводиться все найденные аномалии;
5. Поле отображения аномалии – в этом поле будет график аномалии с выделенным розовым цветом аномальным участком, при этом нормальные участки выделяются синим. Выбрать аномалию для отображения можно в списке 4, нажав на нее два раза или нажав на кнопку «показать аномалию»;
6. Информация об аномалии – в этом поле отображается подробная информация про параметр, а также временные участки, признанные аномалиями;

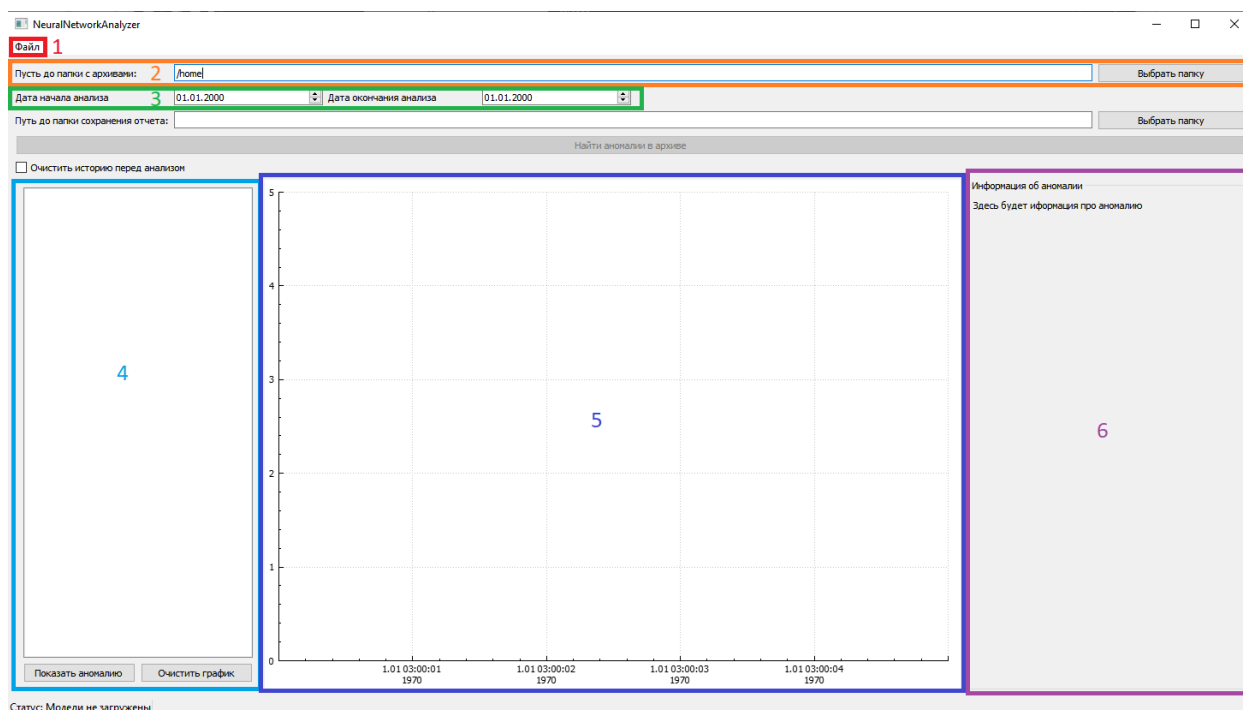


Рисунок 6.2 – Интерфейс программы анализа

Внизу окна располагается строка состояния, из которой можно понять в каком состоянии находится программа. Если необходимо провести анализ еще раз, то можно очистить результат предыдущего анализа, нажав на поле «очистить историю перед анализом». Если этого не сделать, то новые аномалии будут добавляться в конец списка.

Модели потребляют большое количество оперативной памяти, поэтому существуют минимальные системные требования для данной программы:

- Операционная система – Windows 7 или выше;
- Оперативная память – 32 Гб;
- 64-разрядный процессор с тактовой частотой 2 ГГц и выше;

После запуска программы необходимо загрузить модели и файлы нормативно справочной информации. Без этого условия анализ будет невозможен.

Далее необходимо выбрать папку с архивами и указать промежуток времени для анализа. После нажатия кнопки «Найти аномалии в архиве» программа начнет последовательно анализировать архивы, выводя результат в список, который отмечен цифрой 4 на рисунке 6.2.

Пример найденной программой аномалии представлен на рисунке 6.3.

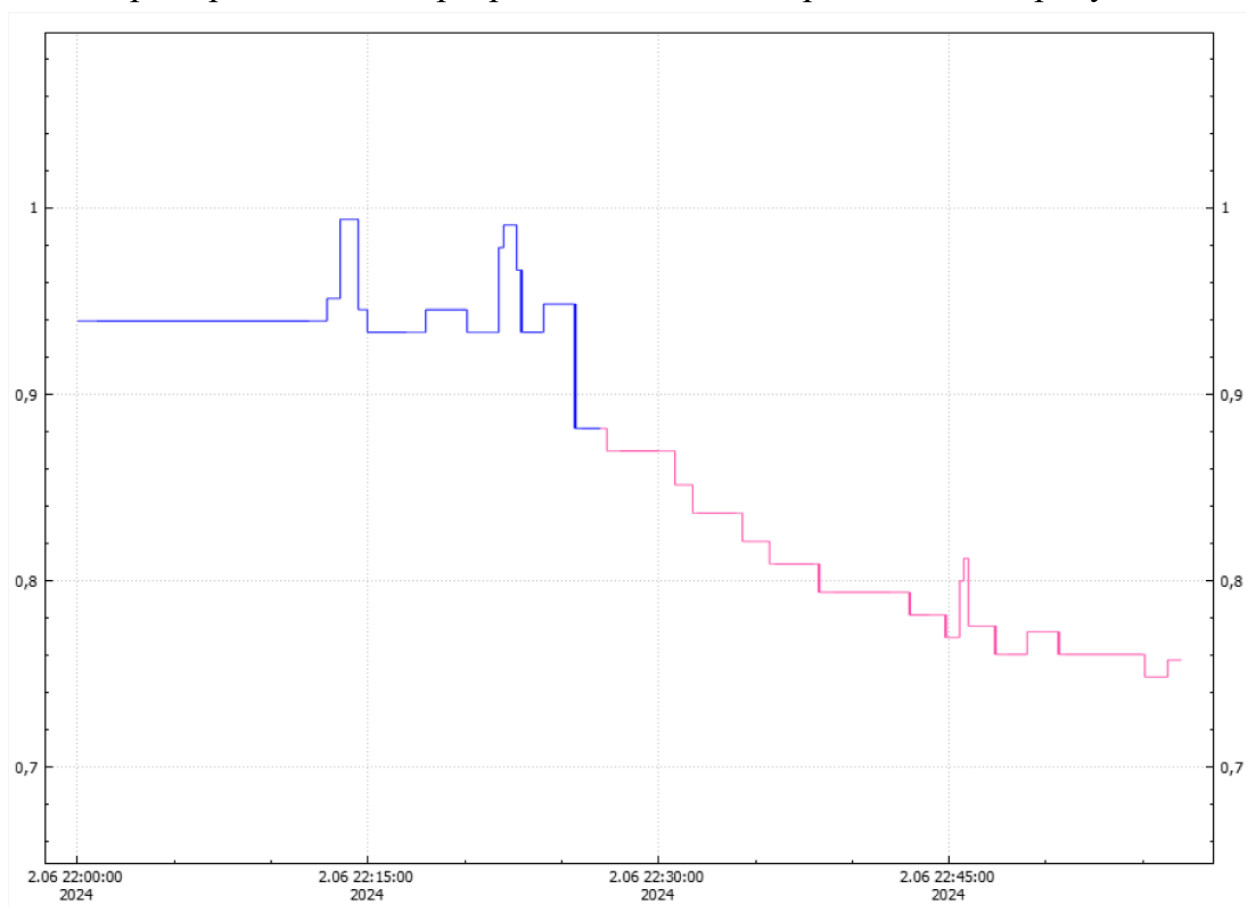


Рисунок 6.3 – Найденная аномалия

На рисунке изображен параметр с идентификатором 139919407. Тип параметра: полюс питания. Из характера графика понятно, что данное устройство полностью вышло из строя, т.к. в нормальном режиме значения держатся на одном уровне с небольшими перепадами значений. Пример нормального поведения был получен при помощи визуализатора архивов и представлен на рисунке 6.4

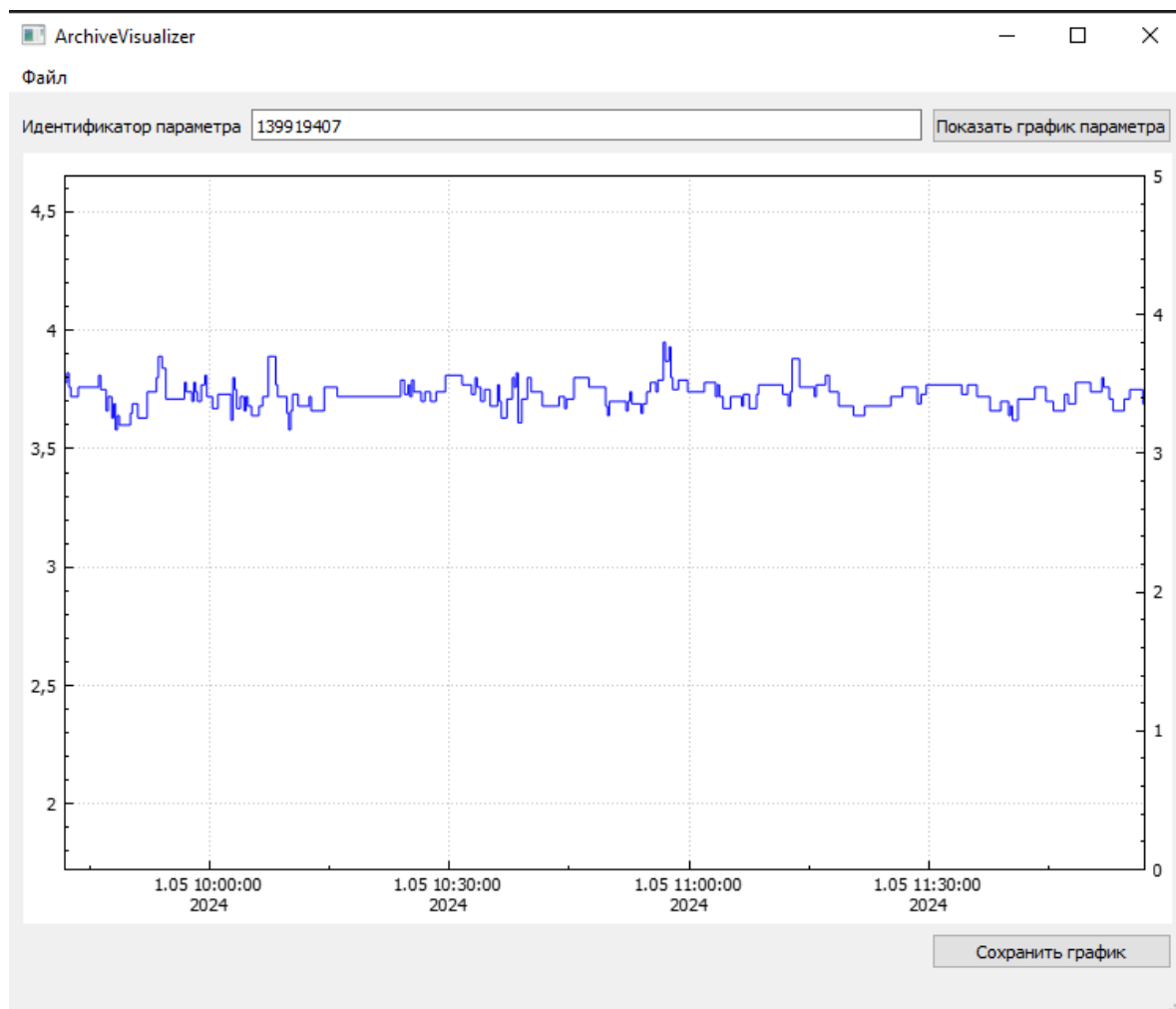


Рисунок 6.4 – Нормальное поведение параметра с идентификатором
139919407

Также при анализе не обошлось и без ложных срабатываний. Пример выявления ложного срабатывания представлен на рисунке 6.5.

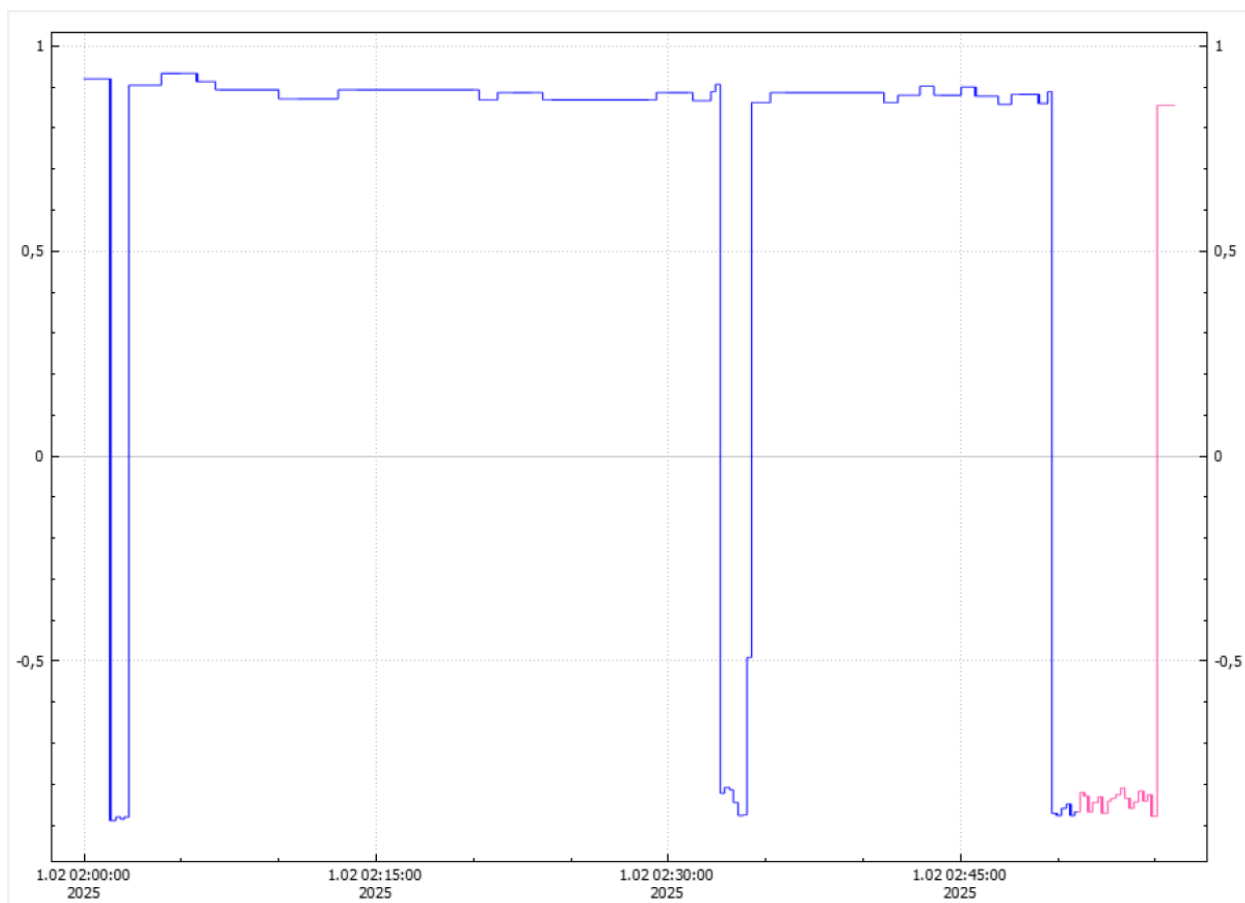


Рисунок 6.5 – Ложное срабатывание

По результатам анализа из архивов за 1 месяц было найдено 20 аномалий, 1 из которых – ложные срабатывания. Т.е. примерно 95% ситуаций программа определяет верно.

7. ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ

7.1. Концепция

Выпускная квалификационная работа посвящена разработке прототипа системы выявления деградационных в динамике изменения параметров работы устройств железнодорожной автоматики и телемеханики. В ходе работы были изучены существующие подходы к решению подобных задач, а также разработано приложение, которое эффективно справляется с поставленной задачей.

Экономическая целесообразность выпускной квалификационной работы бакалавра оценивалась по результатам выполнения следующих пунктов:

- составление плана выполнения работ;
- оценка размеров заработной платы и социальных отчислений участников выполнения проекта;
- расчет материальных затрат;
- расчет амортизационных отчислений;
- расчет полной себестоимости разработки.

Раздел завершается выводами об экономической целесообразности реализации проекта выпускной квалификационной работы бакалавра.

7.2. Составление плана выполнения работ.

Для расчета затрат на этапе проектирования необходимо определить трудоемкость каждого этапа работы. Продолжительность работ измеряется в человеко-днях. Данные представлены в таблице 7.1.

Таблица 7.1 – Продолжительность работ

№	Наименование работы	Длительность работы, чел./дней	
		Руководитель	Исполнитель
1	Формирование требований	1	3
2	Обзор аналогов	1	4
3	Разработка приложения	1	30

Продолжение таблицы 7.1

№	Наименование работы	Длительность работы, чел./дней	
		Руководитель	Исполнитель
4	Тестирование и отладка приложения	1	5
5	Подготовка экономического обоснования работы	1	2
6	Оформление пояснительной записки	2	10
7	Подготовка иллюстративного материала	1	5

Таким образом, время, затраченное на выполнение работы студентом, составляет 59 дней; время, затраченное на выполнение работы научным руководителем, составляет 8 дней.

7.3. Расчет расходов на оплату труда.

Для того, чтобы вычислить объем расходов на оплату труда, был выполнен расчет дневных ставок участников разработки. Согласно Приложению №7 к приказу № ОД/0432 от 15.10.2020 «Положение об оплате труда работников университета» (действующая редакция с 01.10.2020) [5], минимальный размер оплаты труда доцента, с ученой степенью кандидата технических наук, составляет 40600 рублей. Средняя заработная плата инженера-программиста с опытом применения библиотеки Qt составляет 136000 рублей [7].

Рассчитаем дневную ставку, исходя из месячного оклада, принимая количество рабочих дней за 21:

$$ЗРД_{\text{рук}} = \frac{40600}{21} = 1933,33 \text{ руб./день},$$

$$ЗРД_{\text{исп}} = \frac{136\,000}{21} = 6476,2 \text{ руб./день},$$

где $ЗРД_{\text{рук}}$ – дневная заработная плата руководителя, $ЗРД_{\text{исп}}$ – дневная заработная плата исполнителя.

Затраты на основную заработную плату руководителя и исполнителя представлены в табл. 7.2.

Таблица 7.2 – Итоговые затраты на основную заработную плату исполнителей

Исполнитель	Оплата, руб./день	Количество дней	Основная оплата за выполнение проекта, руб
Руководитель	1933,33	8	15 466,64
Студент	6476,2	59	382 084
Итого:			397 550,64

Исходя из затрат на основную заработную плату каждого участника проекта, определим затраты на дополнительную заработную плату по формуле:

$$З_{\text{доп.з./пл}} = З_{\text{осн.з./пл}} \frac{Н_{\text{доп}}}{100} \quad (7.1)$$

где $З_{\text{доп.з./пл}}$ - расходы на дополнительную заработную плату исполнителей (руб.), $З_{\text{осн.з./пл}}$ - расходы на основную заработную плату исполнителей (руб.), $Н_{\text{доп}}$ - норматив дополнительной заработной платы, в рамках данной работы принимается равным 8%.

Расчет расходов на дополнительную заработную плату в соответствии с формулой 7.1 и совокупной величины заработной платы исполнителей представлен в таблице 7.3.

Таблица 7.3 – Итоговые затраты на дополнительную заработную плату исполнителей

Исполнитель	Оплата, руб./день	Количество дней	Дополнительная оплата за выполнение проекта, руб
Руководитель	154,67	8	1 237,36
Студент	518,1	59	30 567,9
Итого:			31 805,26

Отчисления во внебюджетные фонды были определены по формуле 7.2.

$$З_{\text{соц}} = \left(З_{\text{осн.з./пл}} + З_{\text{доп.з./пл}} \right) \cdot \frac{Н_{\text{соц}}}{100}, \quad (7.2)$$

где $Z_{\text{соц.}}$ – отчисления на социальные нужды с заработной платы (руб.),
 $Z_{\text{осн.з./пл}}$ – расходы на основную заработную плату исполнителей (руб.),
 $Z_{\text{доп.з./пл}}$ – расходы на дополнительную заработную плату исполнителей (руб.),
 $N_{\text{соц}}$ – норматив отчислений на страховые взносы на обязательное социальное, пенсионное и медицинское страхование (%), в рамках данной работы принимается равным 30 %.

Затраты социальных отчислений для руководителя рассчитываются как

$$Z_{\text{соц.}} = (1933,33 \text{ руб.} + 154,67 \text{ руб.}) * \frac{30}{100} = 626,4 \text{ руб.}$$

Затраты социальных отчислений для дипломанта рассчитываются как

$$Z_{\text{соц.}} = (6476,2 \text{ руб.} + 518,1 \text{ руб.}) * \frac{30}{100} = 2\,098,29 \text{ руб.}$$

Результаты расчёта отчислений на социальные нужды для руководителя и дипломанта приведены в табл. 7.4.

Таблица 7.4 – Отчисления на социальные нужды

Исполнитель	Оплата, руб./день	Количество дней	Отчисления на социальные нужды, руб
Руководитель	626,4	8	5 011,2
Студент	2098,29	59	123 799,11
Итого:			128 810,31

7.4. Амортизационные отчисления

При разработке были использованы персональный компьютер стоимостью 100000 рублей, принтер лазерный HP Color LaserJet Enterprise M554dn стоимостью 70000 рублей.

Согласно постановлению Правительства РФ от 01.01.2002 №1 (ред. от 27.12.2019) «О Классификации основных средств, включаемых в амортизационные группы» [8], персональные компьютеры и печатающие устройства относятся ко второй группе со сроком полезного использования от

2-х до 3-х лет. Примем нормативный срок полезного использования данных устройств за 3 года. Годовая норма амортизации определяется по формуле 7.3:

$$H_a = \frac{100}{H_{nc}} \%, \quad (7.3)$$

где H_a - годовая норма амортизации, H_{nc} – нормативный срок полезного использования.

$$H_a = \frac{100}{3} = 33,3\%$$

Амортизационные отчисления по основному средству i за год определяются по формуле 7.4:

$$A_i = Ц_{п.н.i} * \frac{H_{ai}}{100}, \quad (7.4)$$

где A_i – амортизационные отчисления за год по i -му основному средству (руб.), $Ц_{п.н.i}$ – первоначальная стоимость i -го основного средства (руб.), H_{ai} – годовая норма амортизации i -го основного средства (%).

$$A_i = (100000 + 70000) * \frac{33,3}{100} = 56\,610 \text{ руб.}$$

Величина амортизационных отчислений по i -му основному средству, используемому при работе над ВКР, определяется по формуле 7.5:

$$A_{iВКР} = A_i * \frac{T_{iВКР}}{12}, \quad (7.5)$$

где $A_{iВКР}$ – амортизационные отчисления по i -му основному средству, используемому при работе над ВКР (руб.); A_i – амортизационные отчисления за год по i -му основному средству (руб.); $T_{iВКР}$ – время, в течение которого студент использует i -ое основное средство (мес.). Время работы составляет 59 дней, приблизительно 2 месяца.

$$A_{iВКР} = 56\,610 * \frac{2}{12} = 9\,435 \text{ руб.}$$

7.5. Расчет материальных затрат

Затраты на сырье и материалы рассчитываются по формуле 7.6 и учитывают также транспортно-заготовительные расходы:

$$З_M = \sum_{l=1}^L G_l C_l \left(1 + \frac{H_{т.з.}}{100}\right), \quad (7.6)$$

где $З_M$ – затраты на сырье и материалы (руб.), L – индекс вида сырья, G_l – норма расхода l -го материала (руб./ед.), $H_{т.з.}$ – норма транспортно-заготовительных расходов, в рамках данной работы принимается равной 10%.

Перечень материалов и расчет расходов на них в соответствии с формулой 7.6 представлен в таблице 7.5.

Таблица 7.5 – Расчет затрат на материалы

Изделие	Тип	Норма расходов на изделие (ед.)	Цена за единицу согласно данным Яндекс.Маркет (руб./шт.)	Сумма на изделие (руб.)
Бумага А4	Формат А4	1	414 [9]	414
Картридж	Лазерный	1	4 307 [10]	4 307
Итого				4 721
Транспортно-заготовительные расходы				472,1
Итого с учетом транспортно-заготовительных расходов				5 193,1

7.6. Оценка накладных расходов.

Накладные расходы – дополнительные к основным затратам расходы, необходимые для обеспечения процессов производства, связанные с управлением, обслуживанием, содержанием и эксплуатацией оборудования плюс ненормированные расходы: брак, штрафы, пеня, проценты и т.д.

Накладные расходы рассчитываются по формуле 7.7:

$$З_{\text{накл.расх.}} = \left(З_{\text{осн.з./пл}} + З_{\text{доп.з./пл}}\right) * \frac{H_{\text{накл.расх.}}}{100}, \quad (7.7)$$

где $З_{\text{накл.расх.}}$ – накладные расходы (руб.), $З_{\text{осн.з./пл}}$ – расходы на основную заработную плату исполнителей (руб.), $З_{\text{доп.з./пл}}$ – расходы на дополнительную заработную плату исполнителей (руб.), $H_{\text{накл.расх.}}$ – процент накладных расходов (%), в рамках данной работы принимается равным 20%.

Таким образом, величина накладных расходов составит:

$$З_{\text{накл.расх.}} = (397\,550,64 \text{ руб.} + 31\,805,26 \text{ руб.}) * \frac{20}{100} = 85\,871,18 \text{ руб.}$$

7.7. Расчет полной себестоимости работы.

Расчет себестоимости выполнения проекта в рамках данной выпускной квалификационной работы в целом, а также расчет структуры себестоимости представлены в таблице 7.6.

Таблица 7.6 – Смета затрат на ВКР

Наименование статьи	Сумма (руб.)	Структура себестоимости, (%)
Материальные затраты	5 193,1	1
Расходы на оплату труда	429 355,9	65
Отчисления на социальные нужды	128 810,31	20
Амортизационные отчисления	9 435	1
Накладные расходы	85 871,18	13
Итого	658 665,49	100

7.8. Вывод

В данном разделе была рассчитана себестоимость программного продукта с точки зрения вложения в него финансовых ресурсов.

Были рассчитаны величины основной и дополнительной заработной платы работников, принимавших участие в проекте, расходы на социальные отчисления, материальные затраты, а также величину амортизационных отчислений за время разработки данного проекта.

Исходя из расчета, представленного в таблице 7.6, затраты на разработку данного проекта составляют 658 665,49 руб.

Основную часть затрат составляют расходы на оплату труда 65%, накладные расходы 13% и отчисления на социальные нужды 20%.

ЗАКЛЮЧЕНИЕ

В ходе проведенного исследования был разработан прототип системы, позволяющий эффективно выявлять отклонения в режиме работы устройств железнодорожной автоматики и телемеханики с использованием алгоритмов машинного обучения. В рамках работы было предложено несколько подходов, а именно классическая нейронная сеть, автоэнкодер, и комбинация автоэнкодера и рекуррентной нейронной сети. Самым эффективным показал себя последний. Такой подход позволил значительно повысить точность диагностики по сравнению с ранее разработанными методами.

Процесс диагностики устройств ЖАТ является критически важным для обеспечения стабильности работы железнодорожного транспорта. В результате реализации разработанной модели удалось минимизировать количество ложных срабатываний до 5% от общего количества выявленных ситуаций, что соответствует стандарту ОАО «РЖД».

Тестирование предложенной системы на реальных данных подтвердило её высокую эффективность в обнаружении аномалий и отклонений в работе оборудования. Однако существенными недостатками являются большие временные затраты на обучение моделей и большое количество дискового пространства, необходимого для хранения обучающих данных. Дальнейшее внедрение этой системы позволит значительно повысить точность диагностики по сравнению с уже разработанными методами.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Федорова В. С., Стригунов В. В. Решение задачи обнаружения аномалий сетевого трафика с помощью сверточной нейронной сети [Электронный ресурс] URL: <https://cyberleninka.ru/article/n/reshenie-odnoy-zadachi-obnaruzheniya-anomaliy-setevogo-trafika-s-pomoschyu-svertochnoy-neyronnoy-seti> (дата обращения: 01.04.2025).

2. Сафронов Д. А., Кацер Ю. Д., Зайцев К. С. Поиск аномалий с помощью автоэнкодеров // International Journal of Open Information Technologies ISSN: 2307-8162 vol. 10, no. 8, 2022 [Электронный ресурс] URL: <https://cyberleninka.ru/article/n/poisk-anomaliy-s-pomoschyu-avtoenkoderov> (дата обращения: 03.04.2025).

3. Зозуля А. Р. Предупреждение неисправностей в системах технического диагностирования и мониторинга устройств железнодорожной автоматики и телемеханики [Электронный ресурс] URL: <https://cyberleninka.ru/article/n/preduprezhdenie-neispravnostey-v-sistemah-tehnicheskogo-diagnostirovaniya-i-monitoringa-ustroystv-zheleznodorozhnoy-avtomatiki-i> (дата обращения: 05.04.2025).

4. Клионский Д.М., Большев А.К. Применение искусственных нейронных сетей в задачах обнаружения аномалий в поведении сложных динамических объектов М.: Радиотехника журнал «Нейрокомпьютеры. Разработка, применение.», 2011 том 26, № 6, с. 107-113.

5. Меньшов М.А. Коэффициент корреляции Пирсона [Электронный ресурс] URL: https://kpfu.ru/portal/docs/F_2064674290/NPS_19.Pirson.Menshov.pdf (дата обращения: 05.04.2025).

6. Положение об оплате труда работников университета [Электронный ресурс]. URL: <https://etu.ru/assets/files/ru/universitet/normativnyedokumenty/Prikaz%20OD432/>

[prilozhenie-7-k-prikazu-nod0342-ot-15.10.2020.pdf](#) (дата обращения: 19.05.2025).

7. Сайт вакансий hh.ru [Электронный ресурс] URL: https://spb.hh.ru/search/vacancy?hhtmFrom=main&hhtmFromLabel=vacancy_search_line&enable_snippets=true&area=2&experience=between1And3&search_field=name&search_field=company_name&search_field=description&text=%D1%81%2B%2B+qt&only_with_salary=true (дата обращения: 19.05.2025).

8. О классификации основных средств, включаемых в амортизационные группы: постановление Правительства Рос. Федерации от 01.01.2002 №1 (ред. 27.12.2019) // Собрание законодательства Российской Федерации.

9. Платформа Яндекс.Маркет, [Электронный ресурс] URL: <https://market.yandex.ru/product--a4-classic-80-g-m/476914202> (дата обращения: 20.05.2025).

10. Платформа Яндекс.Маркет, [Электронный ресурс] URL: <https://market.yandex.ru/product--kartridzh-cactus-212a-w2123a-purpurnyi-cs-w2123a/1828916646> (дата обращения: 20.05.2025).